

Design and Analysis of Algorithm

Advanced Data Structure **(Red Black Tree)**

(Properties, Rotation and Insertion)

LECTURE 19-21

Overview

- A variation of binary search trees.
- Balanced: height is $O(\lg n)$, where n is the number of nodes.
- Operations will take $O(\lg n)$ time in the worst case.

Red Black Tree

- A red-black tree is a binary search tree + 1 bit per node: an attribute color, which is either red or black.
- All leaves are empty (nil) and colored black.
 - A single sentinel, $\text{nil}[T]$, is used for all the leaves of red-black tree T .
 - Color of $[\text{nil}[T]]$ (i.e. leaf node) is black.
 - The root's parent is also $\text{nil}[T]$.
- All other attributes of binary search trees are inherited by red-black trees (i.e. key, left, right, and p as parent).
- We don't care about the key in $\text{nil}[T]$.

Red Black Tree

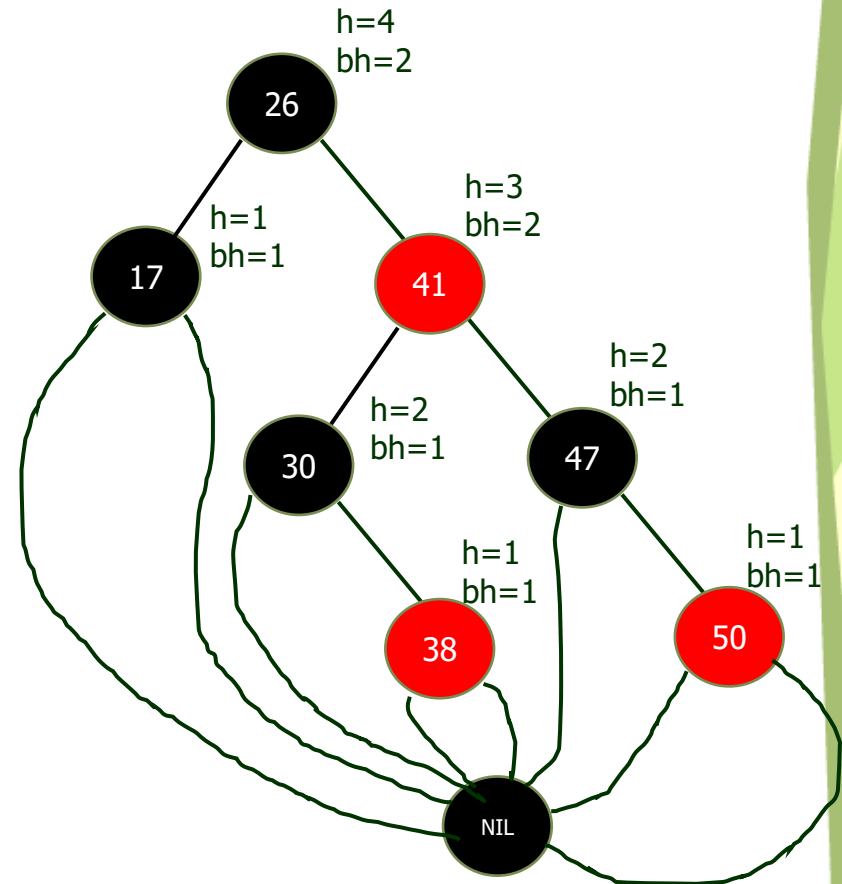
- **Properties:**

1. Every node is either red or black.
2. The root is always black.
3. Every leaf ($\text{nil}[T]$) is black.
4. If a node is red, then both its children are black.
(Hence no two reds in a row on a simple path from the root to a leaf is allowed .)
5. For each node, all paths from the node to descendant leaves contain the same number of black nodes.

Red Black Tree

Height of a red-black tree

- Height of a node is the number of edges in a longest path to a leaf.
- Black-height of a node x : $bh(x)$ is the number of black nodes (including $nil[T]$) on the path from x to leaf, not counting x . By property 5, black-height is well defined.
- So, a red-black tree of height h has **black height $\geq h/2$** . Height of a red-black tree with n nodes is **$h \leq 2 \log_2(n + 1)$**



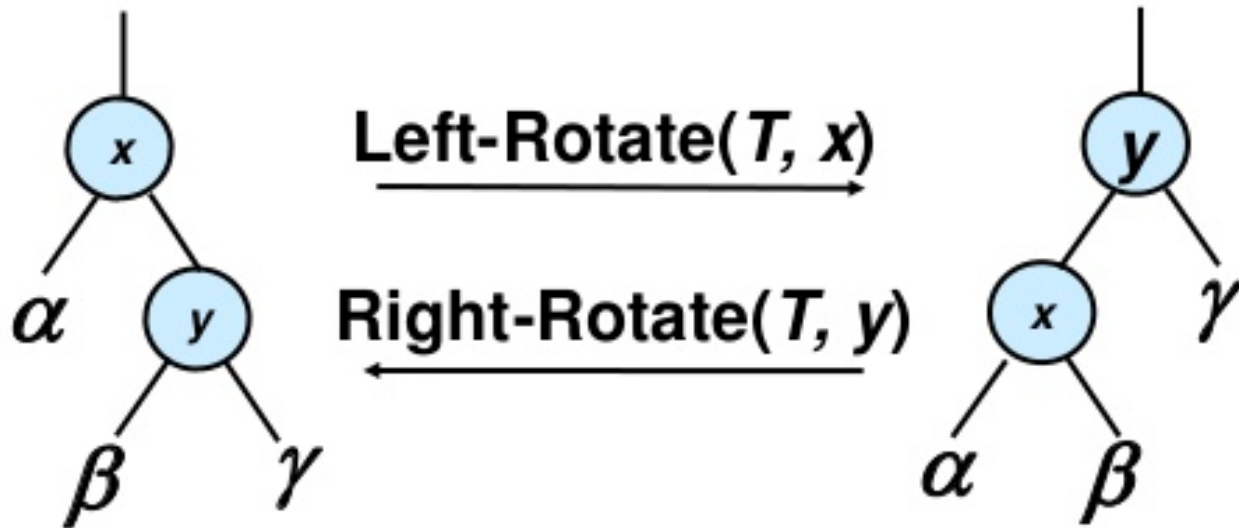
Red Black Tree

Rotations

- The basic tree-restructuring operation.
- Needed to maintain red-black trees as balanced binary search trees.
- Changes the local pointer structure. (Only pointers are changed.)
- Won't upset the binary-search-tree property.
- Have both left rotation and right rotation. They are inverses of each other.
- A rotation takes a red-black-tree and a node within the tree.

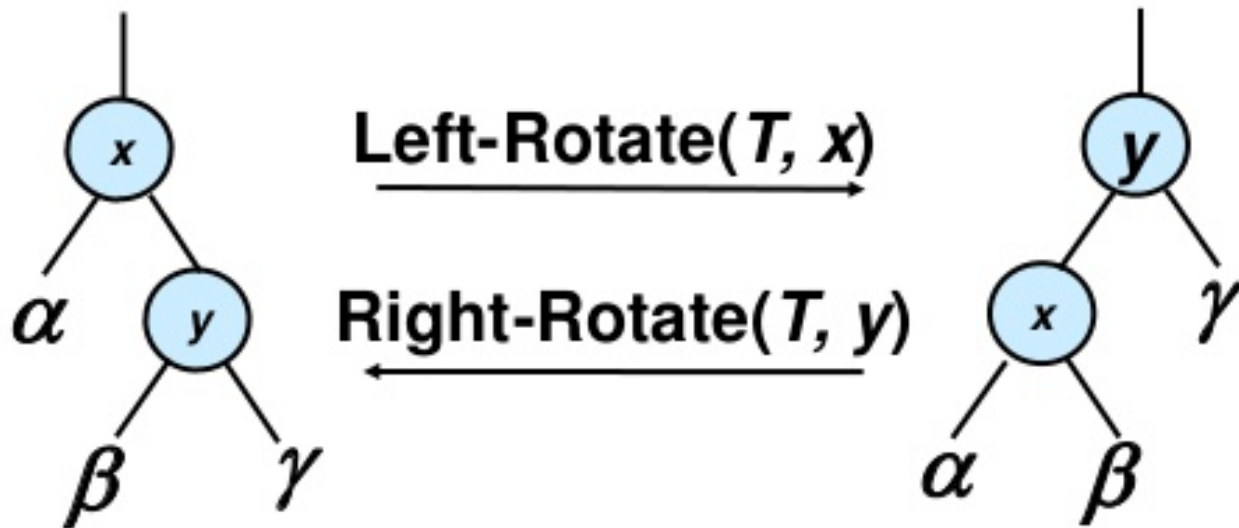
Red Black Tree

Rotations



Red Black Tree

Rotations



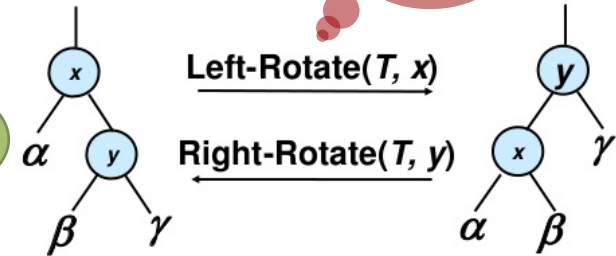
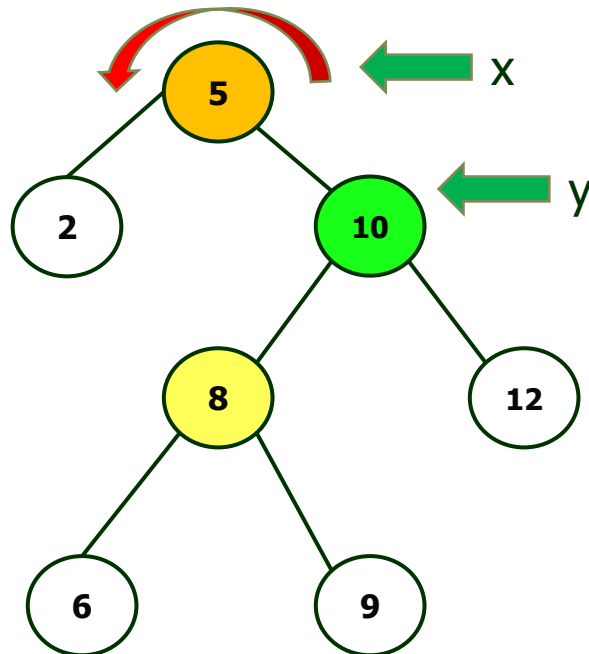
Let's learn the game of changing pointer positions

Red Black Tree

Keep this in mind

Rotations

Left rotate
on 5

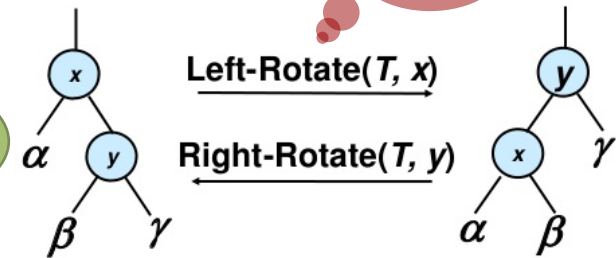
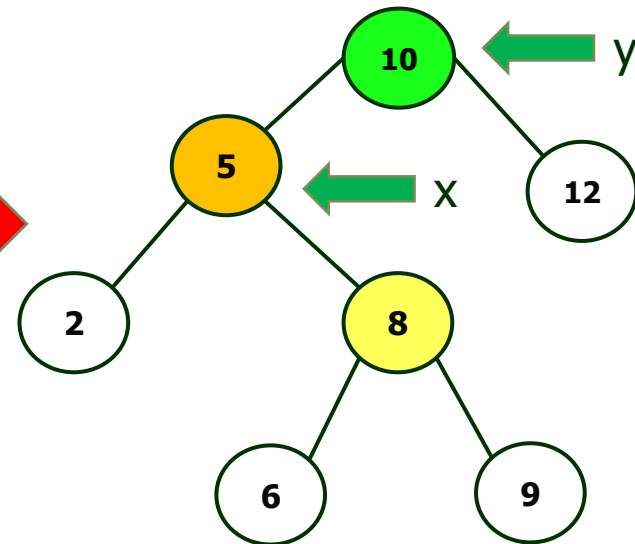
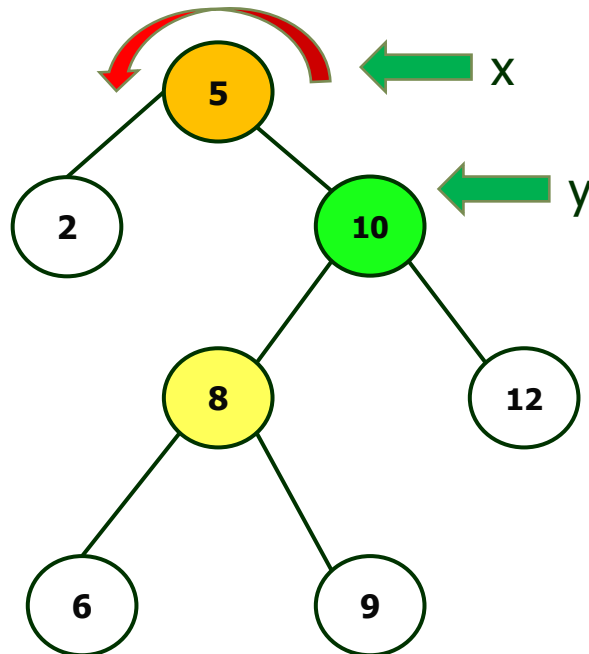


Red Black Tree

Keep this in mind

Rotations

Left rotate
on 5

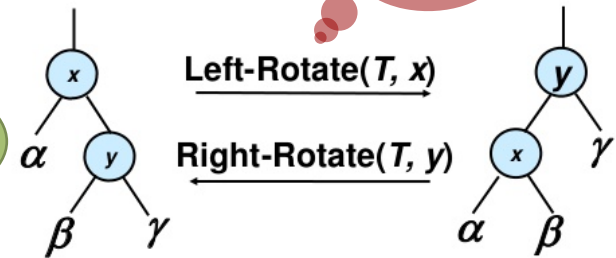
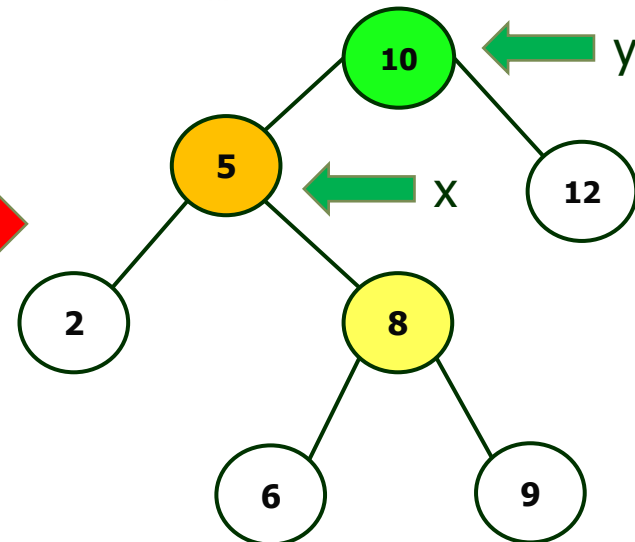
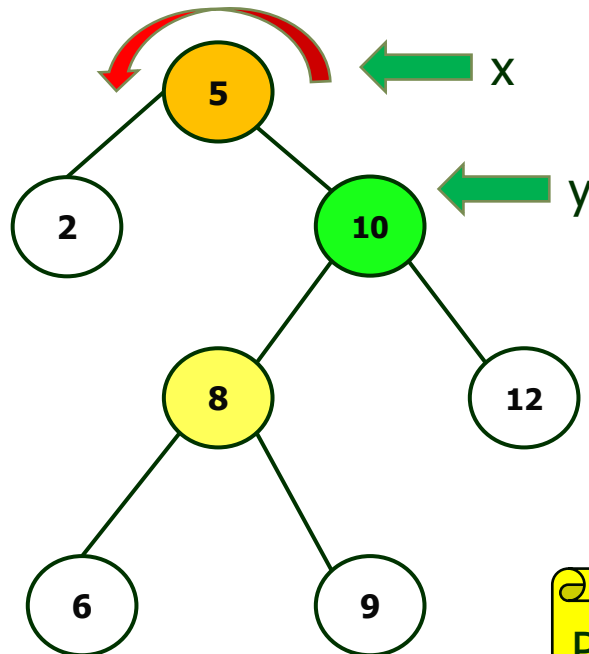


Red Black Tree

Keep this in mind

Rotations

Left rotate
on 5



Please note
down the
observations

Red Black Tree

Rotations

LEFT-ROTATE(T, x)

$y \leftarrow \text{right}[x]$ //Set y .

$\text{right}[x] \leftarrow \text{left}[y]$ //Turn y 's left subtree into x 's right subtree.

if $\text{left}[y] \neq \text{nil}[T]$

then $p[\text{left}[y]] \leftarrow x$

$p[y] \leftarrow p[x]$ //Link x 's parent

if $p[x] = \text{nil}[T]$

then $\text{root}[T] \leftarrow y$

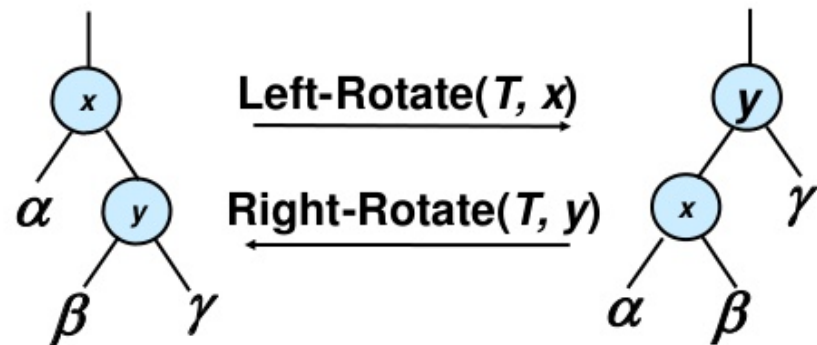
else if $x = \text{left}[p[x]]$

then $\text{left}[p[x]] \leftarrow y$

else $\text{right}[p[x]] \leftarrow y$

$\text{left}[y] \leftarrow x$ //Put x on y 's left.

$p[x] \leftarrow y$



Red Black Tree

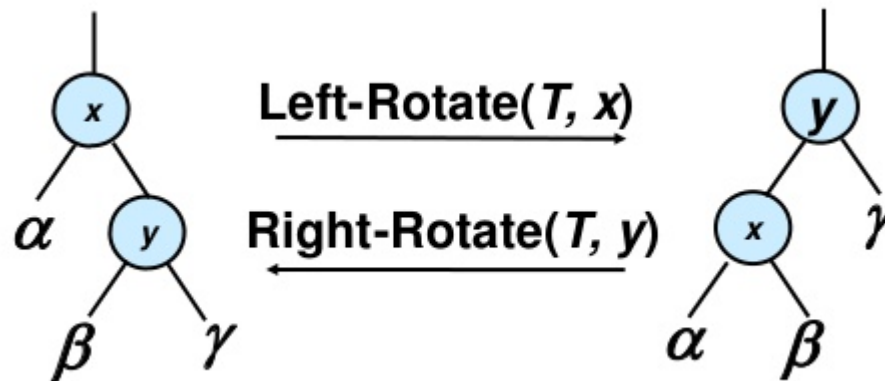
Rotations

The pseudocode for LEFT-ROTATE assumes that

$right[x] = nil[T]$, and

root.s parent is $nil[T]$.

Pseudocode for RIGHT-ROTATE is symmetric: exchange *left* and *right* everywhere.



Red Black Tree

Rotations

RIGHT-ROTATE(T, x)

$y \leftarrow \text{left}[x]$ //Set y .

$\text{left}[x] \leftarrow \text{right}[y]$ //Turn y 's left subtree into x 's right subtree.

if $\text{right}[y] \neq \text{nil}[T]$

then $p[\text{right}[y]] \leftarrow x$

$p[y] \leftarrow p[x]$ //Link x 's parent

if $p[x] = \text{nil}[T]$

then $\text{root}[T] \leftarrow y$

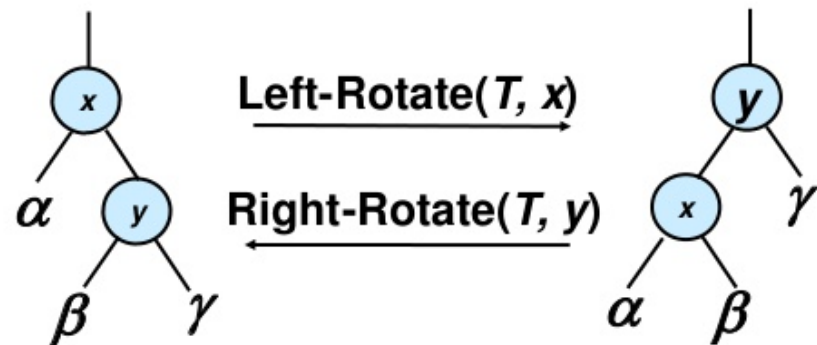
else if $x = \text{right}[p[x]]$

then $\text{right}[p[x]] \leftarrow y$

else $\text{left}[p[x]] \leftarrow y$

$\text{right}[y] \leftarrow x$ //Put x on y 's right.

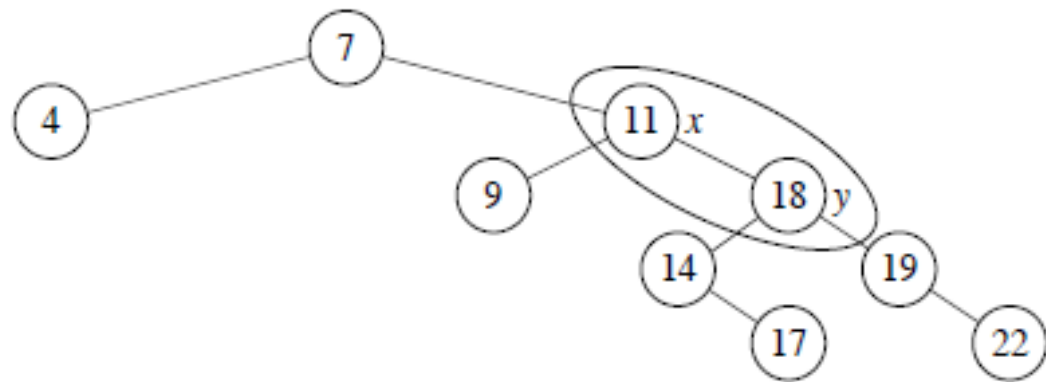
$p[x] \leftarrow y$



Red Black Tree

Rotations (Example)

Demonstrate of left rotation that maintains in-order ordering of keys.

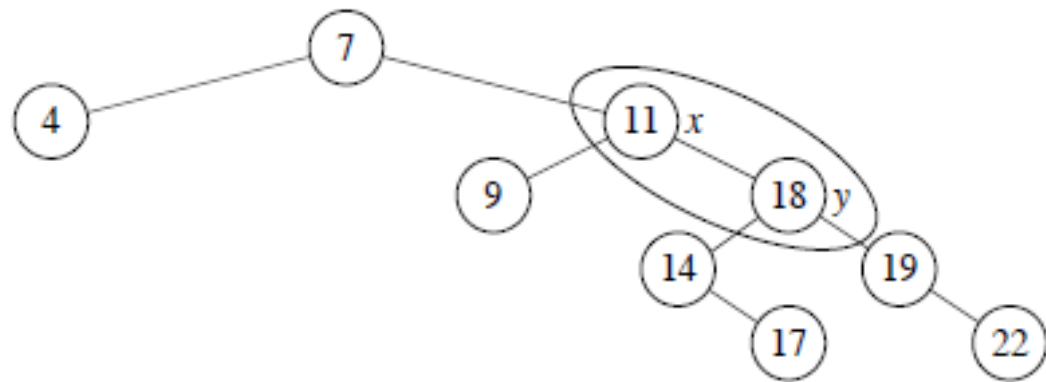


Red Black Tree

Rotations (Example)

Demonstrate of left rotation that maintains in-order ordering of keys.

LEFT-ROTATE(T, x)

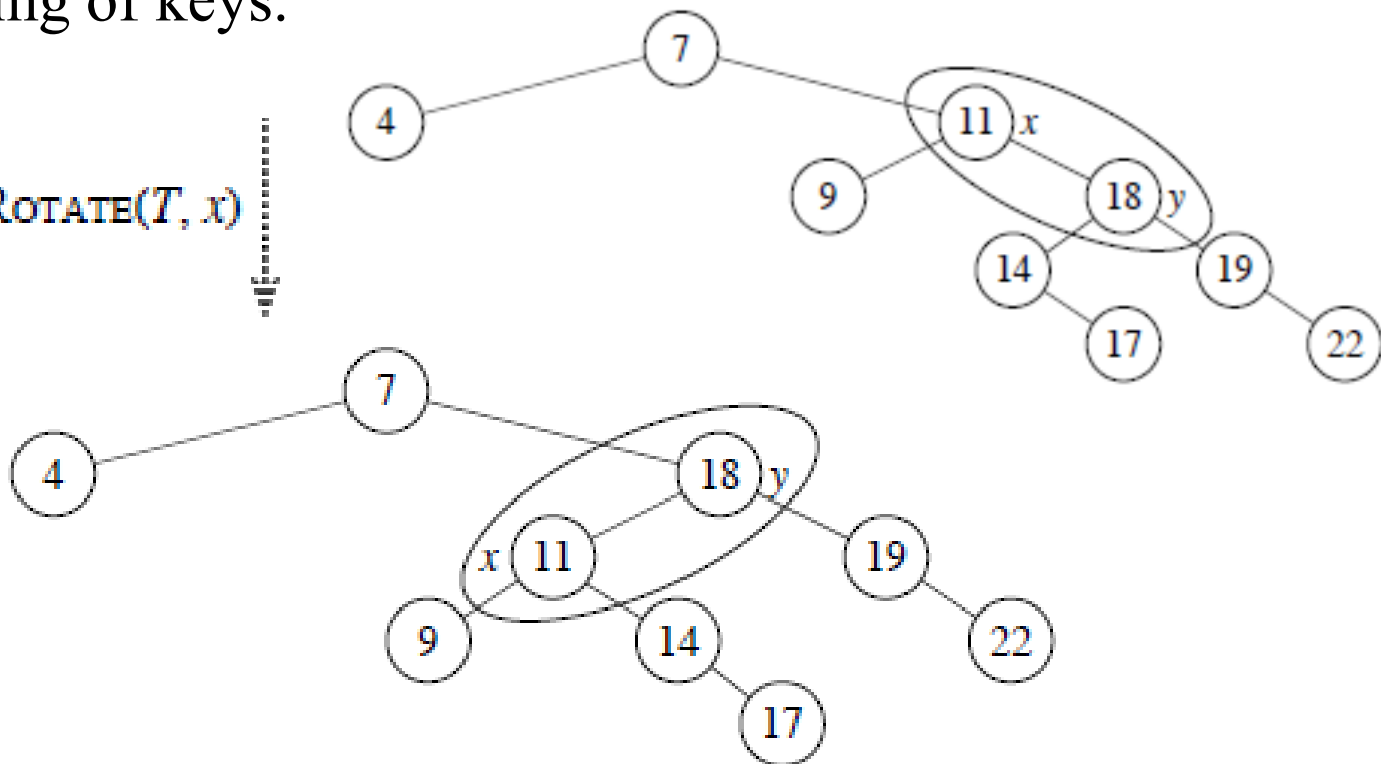


Red Black Tree

Rotations (Example)

Demonstrate of left rotation that maintains in-order ordering of keys.

LEFT-ROTATE(T, x)

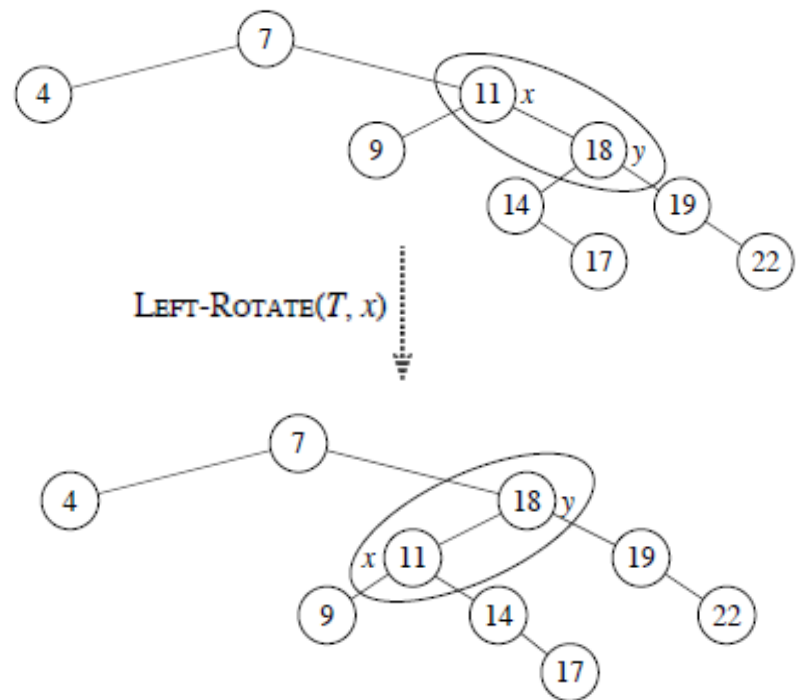


Red Black Tree

Rotations (Example)

Demonstrate of left rotation that maintains in-order ordering of keys.

- Before rotation: keys of x 's left subtree $\leq 11 \leq$ keys of y 's left subtree $\leq 18 \leq$ keys of y 's right subtree.
- Rotation makes y 's left subtree into x 's right subtree.
- After rotation: keys of x 's left subtree $\leq 11 \leq$ keys of x 's right subtree $\leq 18 \leq$ keys of y 's right subtree.
- Time complexity : $O(1)$ for both LEFT-ROTATE and RIGHT-ROTATE, since a constant number of pointers are modified.



Red Black Tree

Insertion:

Start by doing regular binary-search-tree insertion:

Red Black Tree

Insertion:

```
RB-INSERT( $T, z$ )  
   $y \leftarrow nil[T]$   
   $x \leftarrow root[T]$   
  while  $x \neq nil[T]$   
    do  $y \leftarrow x$   
      if  $key[z] < key[x]$   
        then  $x \leftarrow left[x]$   
        else  $x \leftarrow right[x]$   
   $p[z] \leftarrow y$   
  if  $y = nil[T]$   
    then  $root[T] \leftarrow z$   
    else if  $key[z] < key[y]$   
      then  $left[y] \leftarrow z$   
      else  $right[y] \leftarrow z$   
   $left[z] \leftarrow nil[T]$   
   $right[z] \leftarrow nil[T]$   
   $color[z] \leftarrow RED$   
  RB-INSERT-FIXUP( $T, z$ )
```

Red Black Tree

Insertion:

```
RB-INSERT( $T, z$ )  
   $y \leftarrow \text{nil}[T]$   
   $x \leftarrow \text{root}[T]$   
  while  $x \neq \text{nil}[T]$   
    do  $y \leftarrow x$   
      if  $\text{key}[z] < \text{key}[x]$   
        then  $x \leftarrow \text{left}[x]$   
        else  $x \leftarrow \text{right}[x]$   
   $p[z] \leftarrow y$   
  if  $y = \text{nil}[T]$   
    then  $\text{root}[T] \leftarrow z$   
    else if  $\text{key}[z] < \text{key}[y]$   
      then  $\text{left}[y] \leftarrow z$   
      else  $\text{right}[y] \leftarrow z$   
   $\text{left}[z] \leftarrow \text{nil}[T]$   
   $\text{right}[z] \leftarrow \text{nil}[T]$   
   $\text{color}[z] \leftarrow \text{RED}$   
  RB-INSERT-FIXUP( $T, z$ )
```

Simply Binary
Search Tree
Implementation

Red Black Tree

Insertion:

- RB-INSERT ends by coloring the new node z red.
- Then it calls RB-INSERT-FIXUP to maintain the properties of a red-black Tree.

Which property might be violated?

1. Every node is either red or black.	OK
2. The root is always black.	If z is the root, then there's a violation. Otherwise, OK.
3. Every leaf ($\text{nil}[T]$) is black.	OK
4. If a node is red, then both its children are black.	If $p[z]$ is red, there's a violation: both z and $p[z]$ are red.
5. For each node, all paths from the node to descendant leaves contain the same number of black nodes.	OK

Red Black Tree

Insertion (RB-INSERT-FIXUP CASE 1) (y is red and z is a right child)

[Nodes with bold outline indicate black nodes and light outline indicate red nodes]

if $color[y] = \text{RED}$

then $color[p[z]] \leftarrow \text{BLACK}$

//Case 1

$color[y] \leftarrow \text{BLACK}$

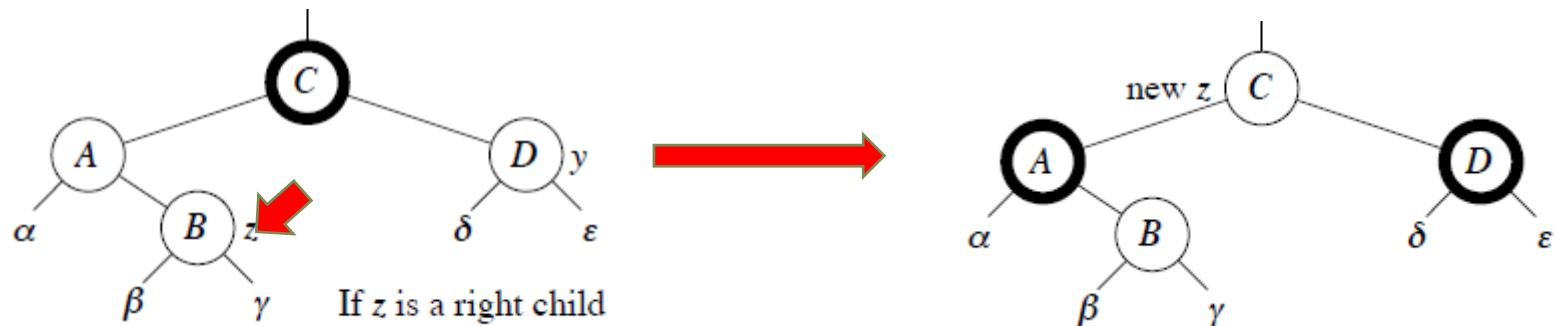
//Case 1

$color[p[p[z]]] \leftarrow \text{RED}$

//Case 1

$z \leftarrow p[p[z]]$

//Case 1



Red Black Tree

Insertion (RB-INSERT-FIXUP CASE 1) (y is red and z is a left child)

[Nodes with bold outline indicate black nodes and light outline indicate red nodes]

if $color[y] = \text{RED}$

then $color[p[z]] \leftarrow \text{BLACK}$

//Case 1

$color[y] \leftarrow \text{BLACK}$

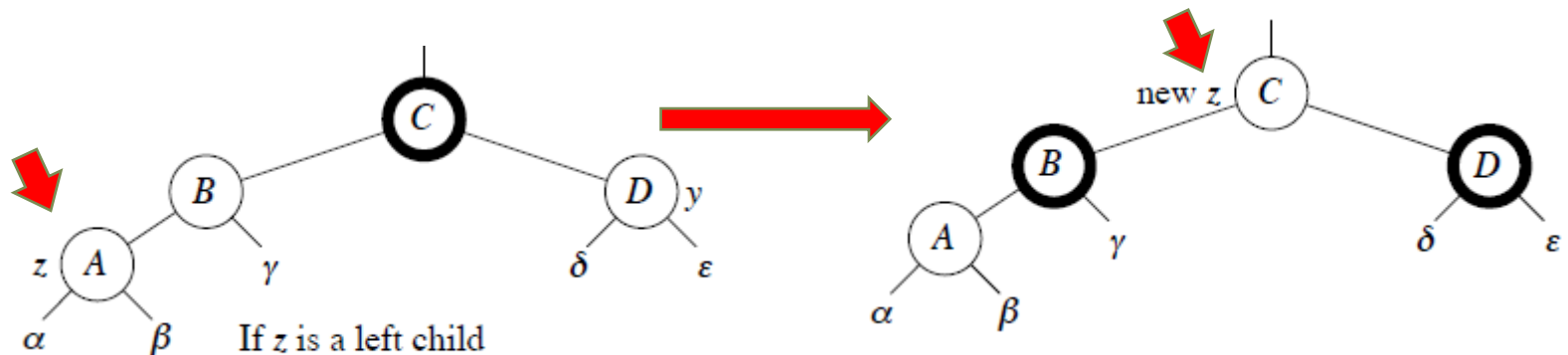
//Case 1

$color[p[p[z]]] \leftarrow \text{RED}$

//Case 1

$z \leftarrow p[p[z]]$

//Case 1



Red Black Tree

Insertion (RB-INSERT-FIXUP CASE 2 (*y is black, z is a right child*)
 [Nodes with bold outline indicate black nodes and light outline indicate red nodes]

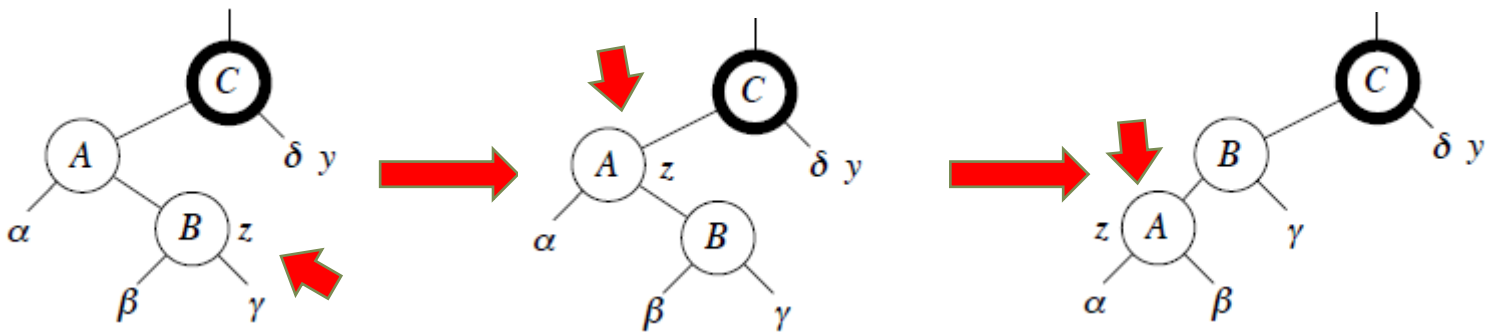
if $z = \text{right}[p[z]]$

then $z \leftarrow p[z]$

LEFT-ROTATE(T, z)

//Case 2

//Case 2



Red Black Tree

Insertion (RB-INSERT-FIXUP CASE 3) (*y is black, z is a left child*)

[Nodes with bold outline indicate black nodes and light outline indicate red nodes]

$color[p[z]] \leftarrow \text{BLACK}$

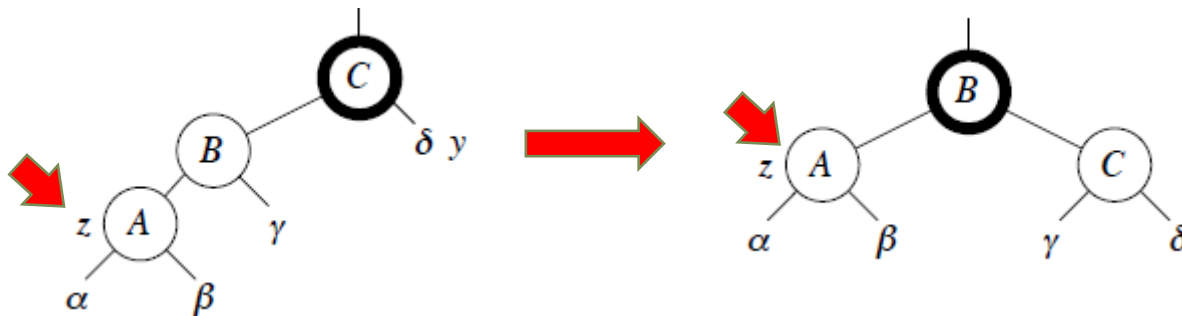
// Case 3

$color[p[p[z]]] \leftarrow \text{RED}$

// Case 3

$\text{RIGHT-ROTATE}(T, p[p[z]])$

// Case 3



Red Black Tree

Insertion:

RB-INSERT-FIXUP(T, z)

while $color[p[z]] = \text{RED}$

do if $p[z] = \text{left}[p[p[z]]]$

then $y \leftarrow \text{right}[p[p[z]]]$

if $color[y] = \text{RED}$

then

Apply Case 1

else if $z = \text{right}[p[z]]$

then

Apply Case 2

Apply Case 3

else (same as **then** clause with *.right.* and *.left.* exchanged)

$color[\text{root}[T]] \leftarrow \text{BLACK}$

Red Black Tree

Insertion:

RB-INSERT-FIXUP(T, z)

while $color[p[z]] = \text{RED}$

do if $p[z] = \text{left}[p[p[z]]]$

then $y \leftarrow \text{right}[p[p[z]]]$

if $color[y] = \text{RED}$

then

Apply Case 1

else if $z = \text{right}[p[z]]$

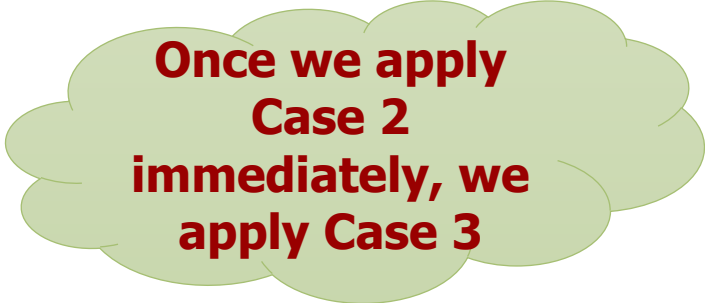
then

Apply Case 2

Apply Case 3

else (same as **then** clause with .right. and .left. exchanged)

$color[\text{root}[T]] \leftarrow \text{BLACK}$



Once we apply
Case 2
immediately, we
apply Case 3

Red Black Tree

Insertion:

RB-INSERT-FIXUP(T, z)

while $color[p[z]] = \text{RED}$

do if $p[z] = \text{left}[p[p[z]]]$

then $y \leftarrow \text{right}[p[p[z]]]$

if $color[y] = \text{RED}$

then $color[p[z]] \leftarrow \text{BLACK}$

$color[y] \leftarrow \text{BLACK}$

$color[p[p[z]]] \leftarrow \text{RED}$

$z \leftarrow p[p[z]]$

else if $z = \text{right}[p[z]]$

then $z \leftarrow p[z]$

 LEFT-ROTATE(T, z)

$color[p[z]] \leftarrow \text{BLACK}$

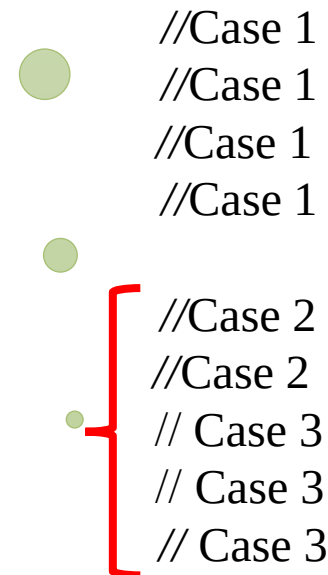
$color[p[p[z]]] \leftarrow \text{RED}$

 RIGHT-ROTATE($T, p[p[z]]$)

else (same as **then** clause with .right. and .left. exchanged)

$color[\text{root}[T]] \leftarrow \text{BLACK}$

Once we apply
Case 2
immediately, we
apply Case 3



Red Black Tree

Insertion (Example):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -11



Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -11



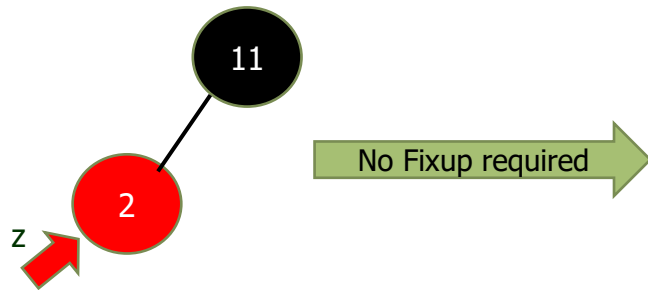
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -2



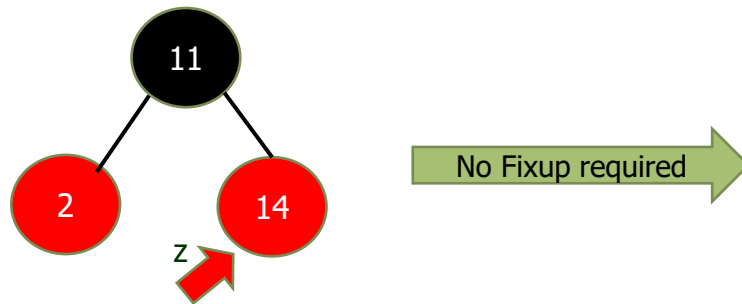
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -14



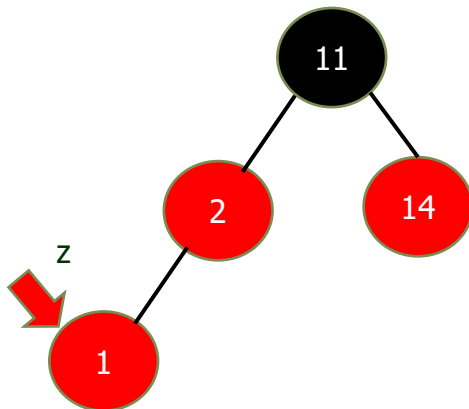
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -1



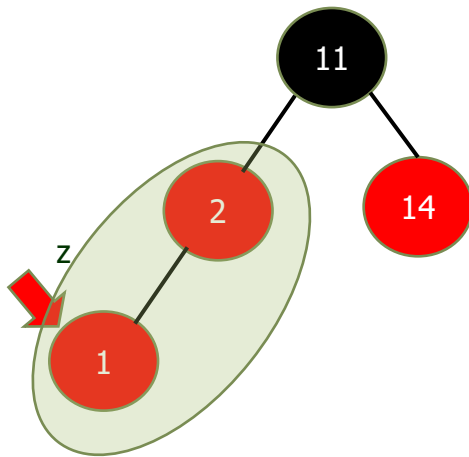
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -1



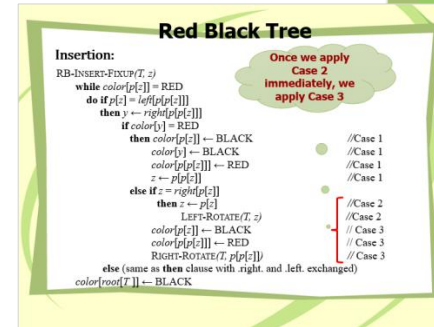
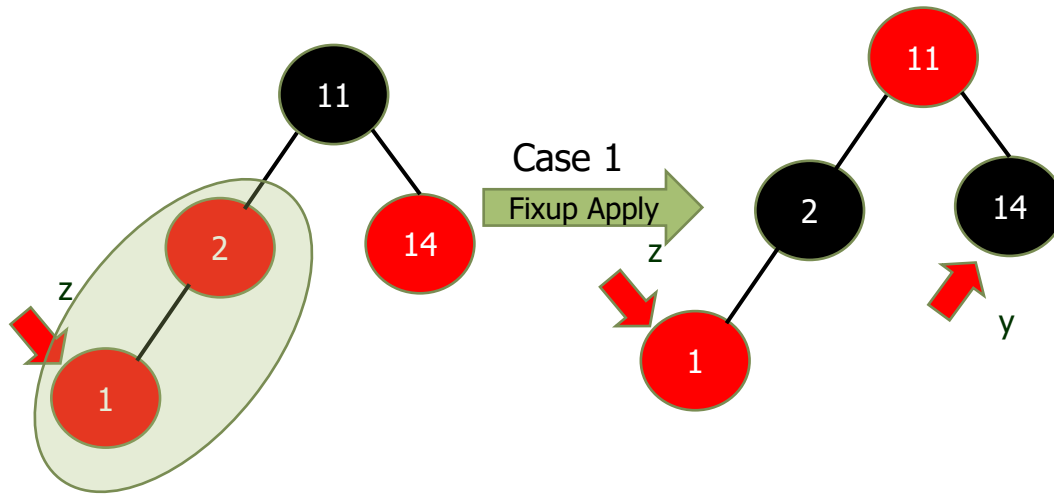
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -1



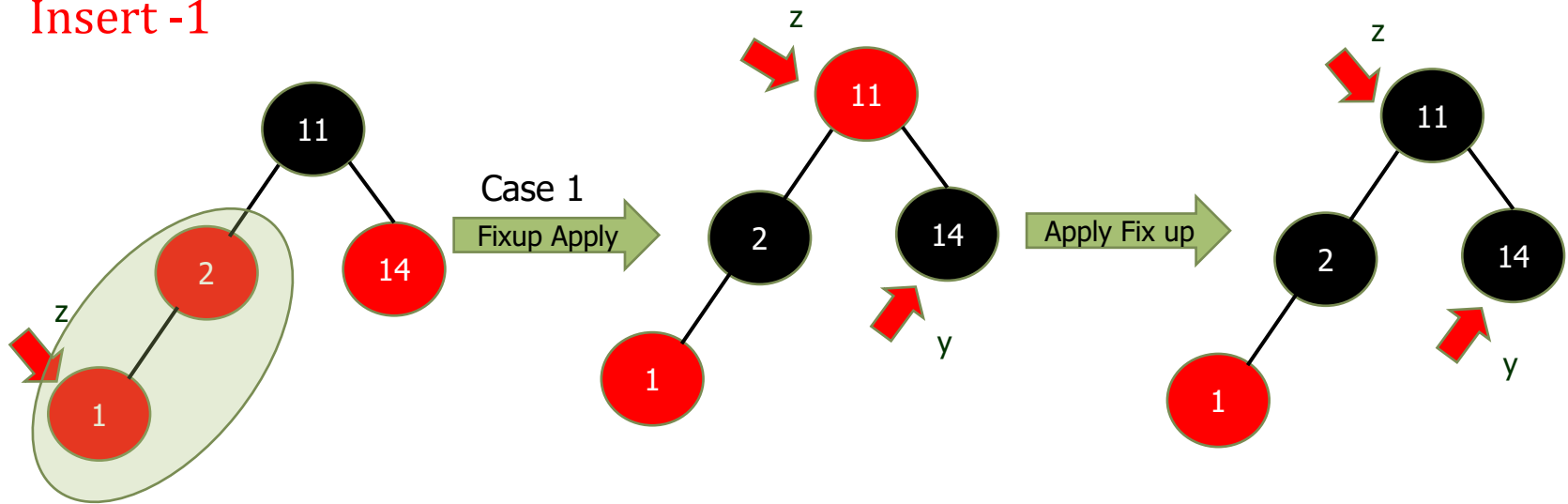
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -1



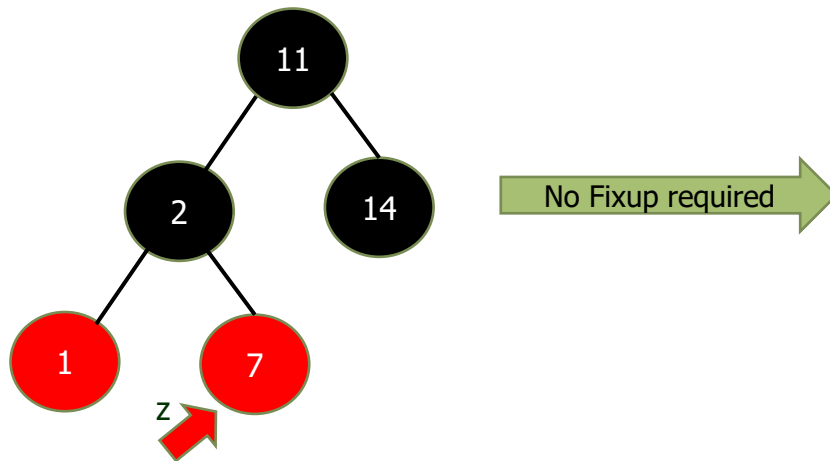
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -7



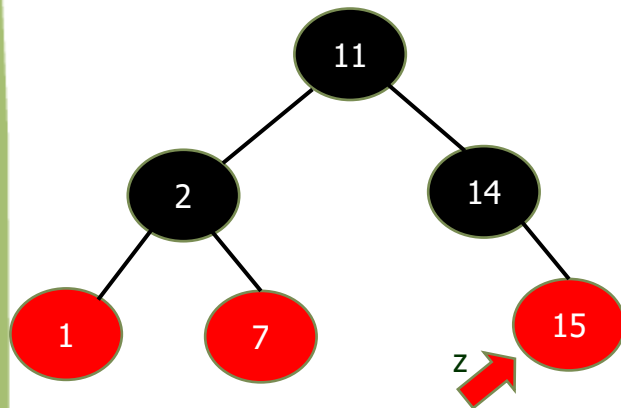
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -15



No Fixup required

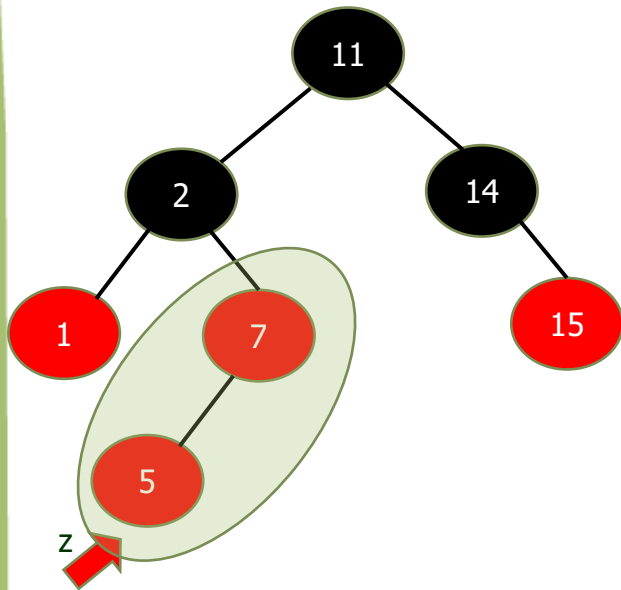
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -5



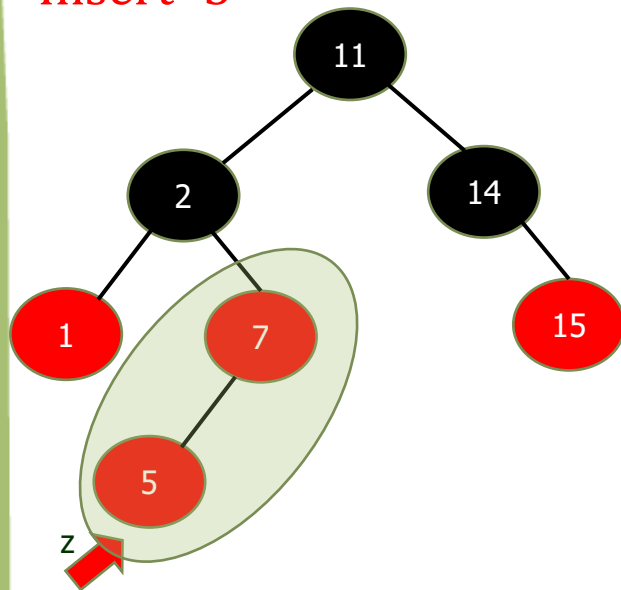
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

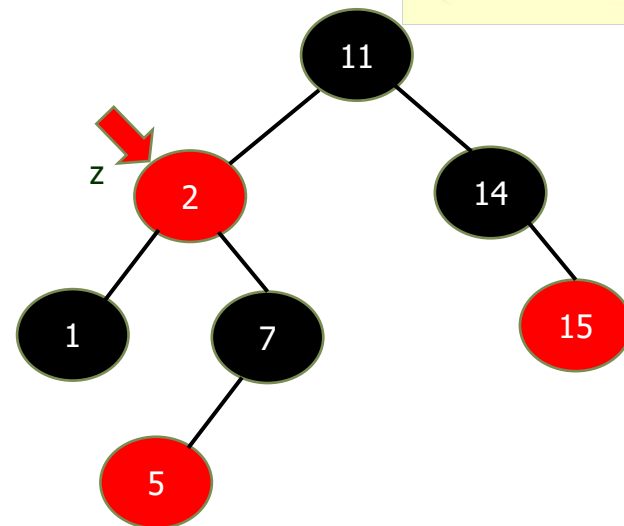
[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -5



Case 1

Fixup Apply



Red Black Tree

Insertion:

RB-INSERT-FIXUP(T, z)

while color[p[z]] = RED

do if p[z] = left[p[p[z]]]

then y ← right[p[p[z]]]

if color[y] = RED

then color[p[z]] ← BLACK

color[y] ← BLACK

color[p[p[z]]] ← RED

z ← p[p[z]]

else if z = right[p[p[z]]]

then z ← p[p[z]]

LEFT-ROTATE(T, z)

color[p[z]] ← BLACK

color[p[p[z]]] ← RED

RIGHT-ROTATE(T, p[p[z]])

else (same as then clause with right and left exchanged)

color[root(T)] ← BLACK

Once we apply

Case 2

immediately, we

apply Case 3

//Case 1

//Case 1

//Case 1

//Case 1

//Case 1

//Case 2

//Case 2

//Case 2

//Case 3

//Case 3

//Case 3

//Case 3

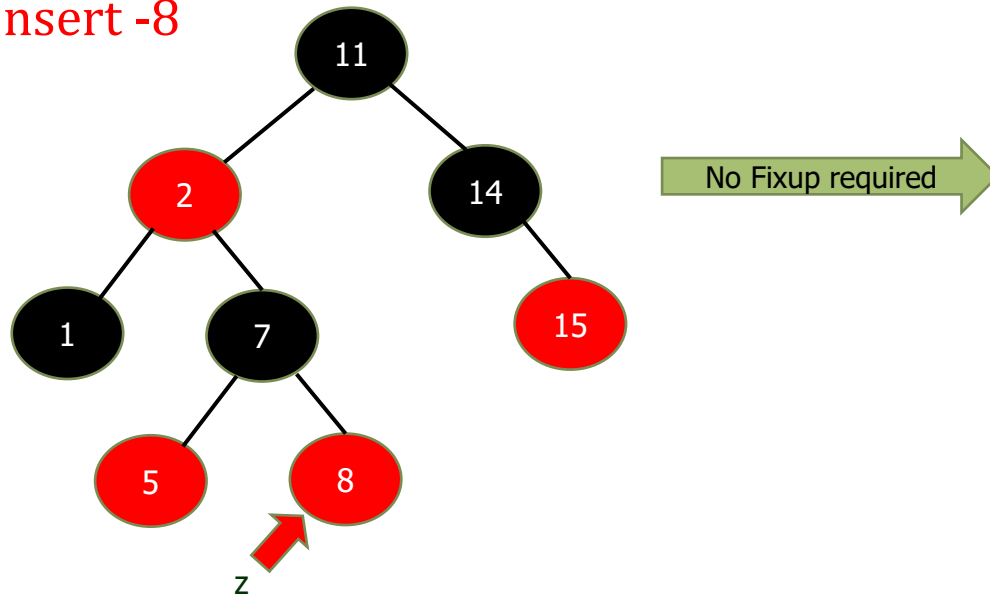
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -8



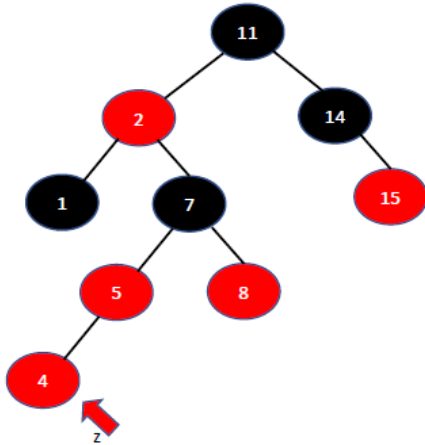
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -4



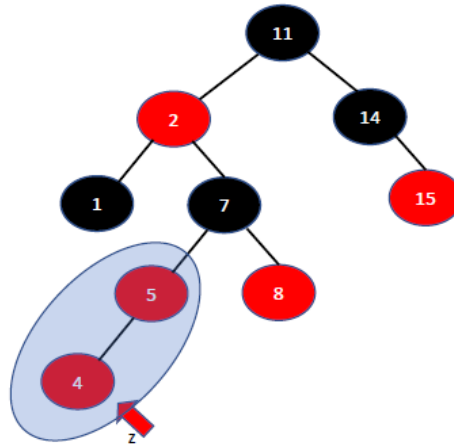
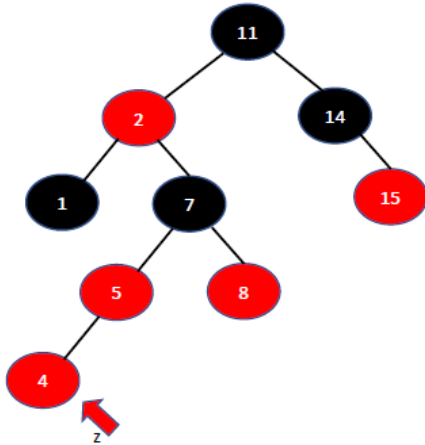
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -4



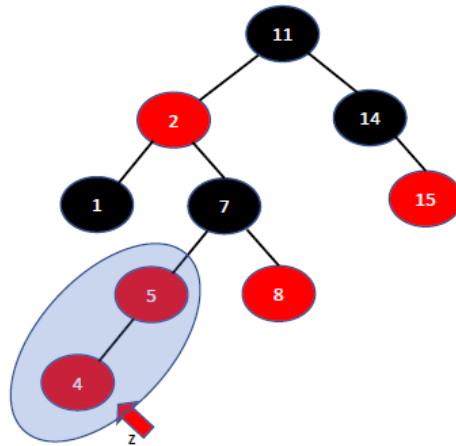
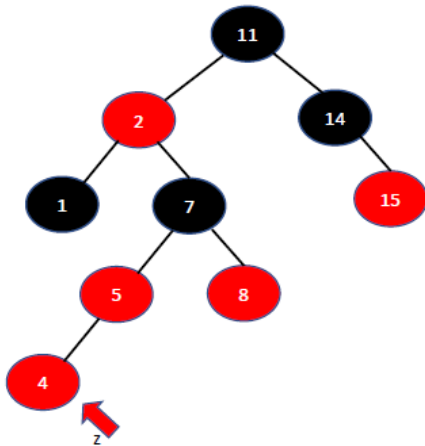
Red Black Tree

Insertion (Example 1):

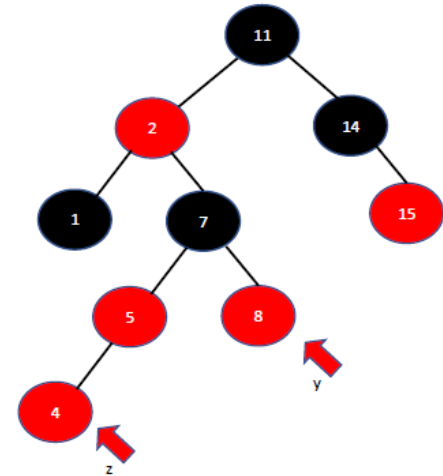
Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -4



Case 1
Fixup Apply



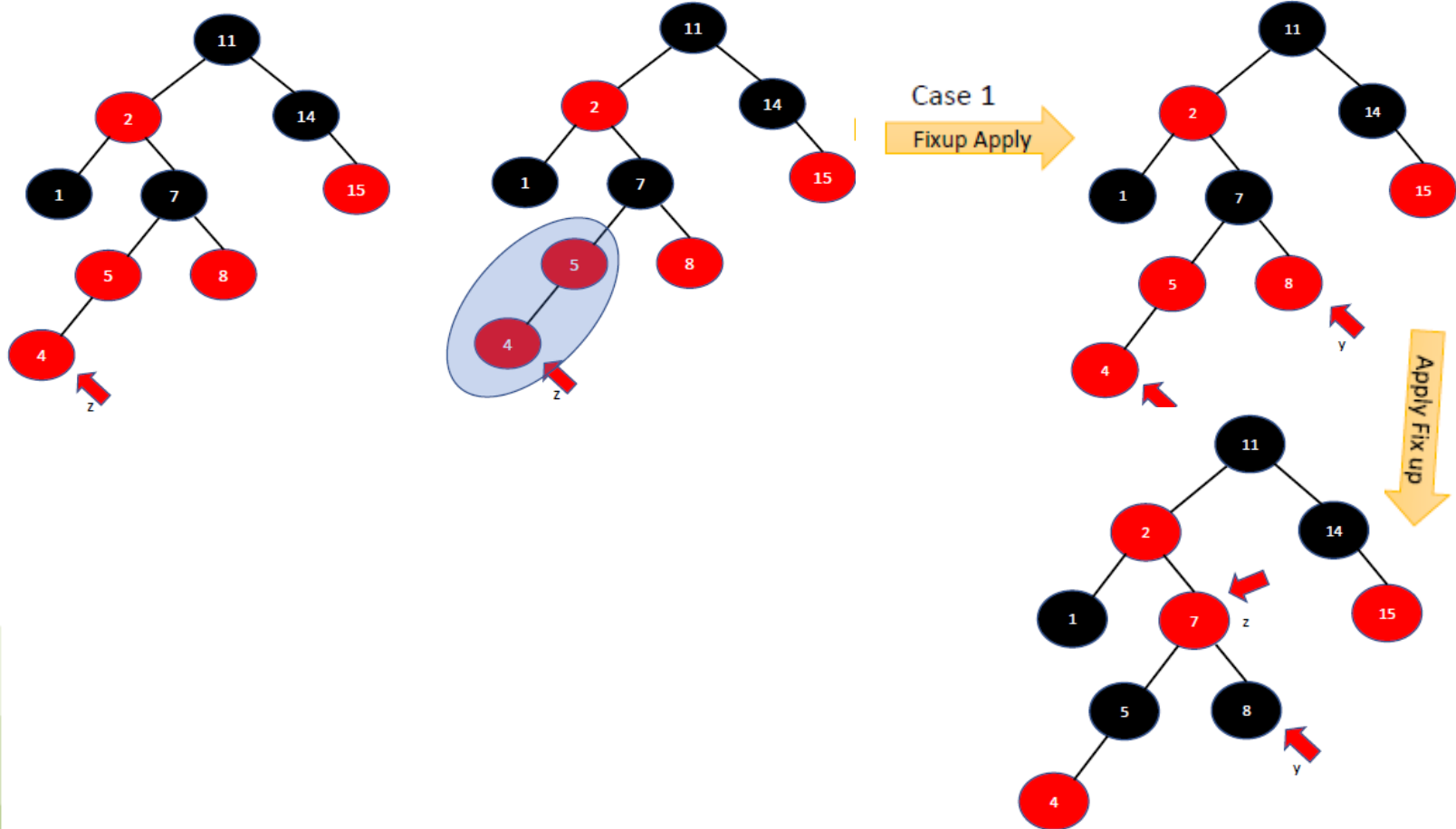
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -4

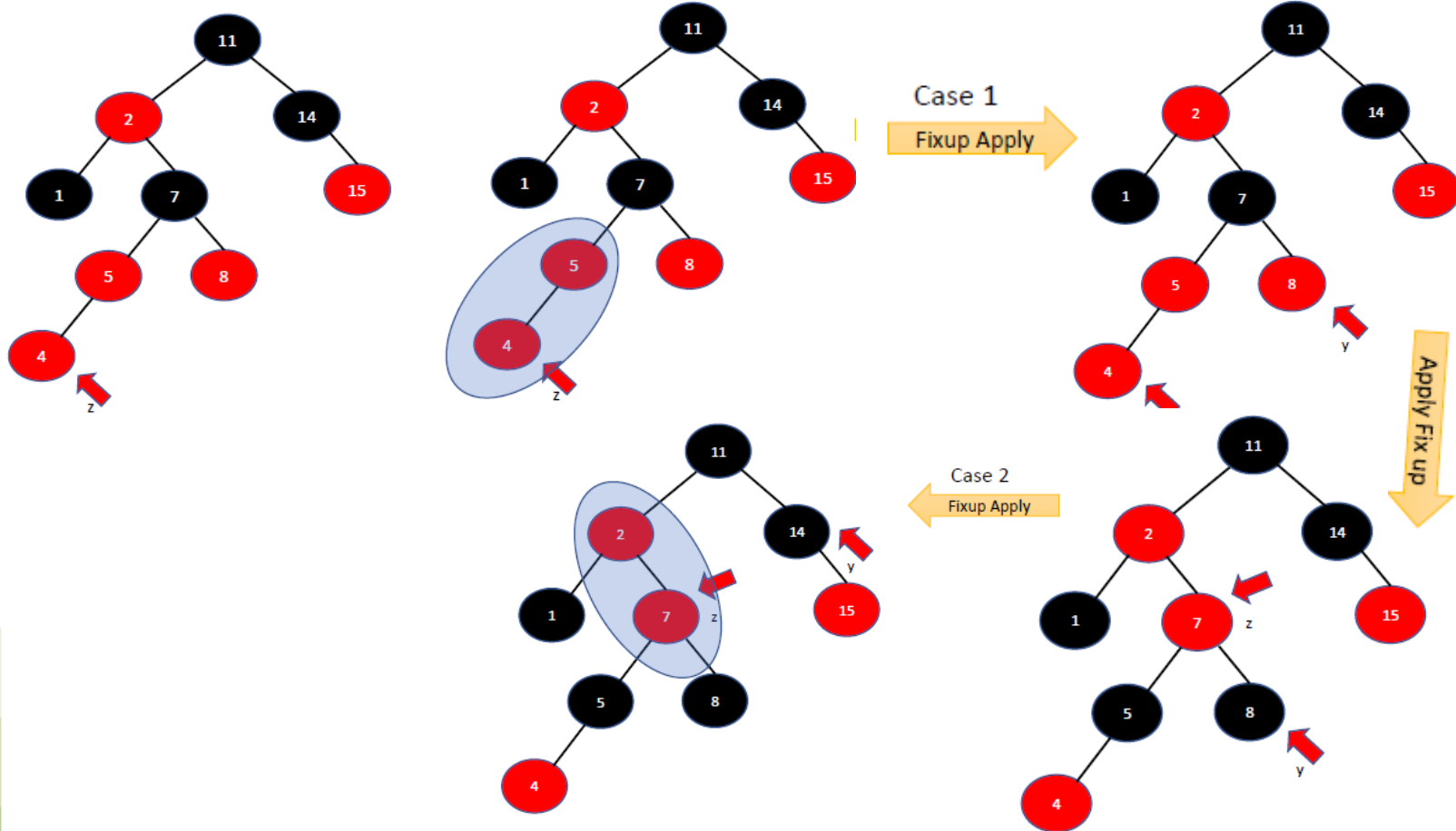


Insertion (Example):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -4



Red Black Tree

Insertion:

```

RB-INSERT-FIXUP(T, z)
while color[p[z]] = RED
do if p[z] = left[p[z]]
then y = right[p[z]]
else y = left[p[z]]
if color[y] = RED
then color[p[z]] = BLACK
color[y] = BLACK
color[p[p[z]]] = RED
z = p[p[z]]
else if z = right[p[z]]
then z = p[z]
LEFT-ROTATE(T, z)
color[p[z]] = BLACK
color[p[p[z]]] = RED
RIGHT-ROTATE(T, p[p[z]])
else (same as then clause with right and left exchanged)
color[root(T)] = BLACK
    
```

Once we apply Case 2 immediately, we apply Case 3

// Case 1
// Case 1
// Case 1
// Case 2
// Case 2
// Case 3
// Case 3

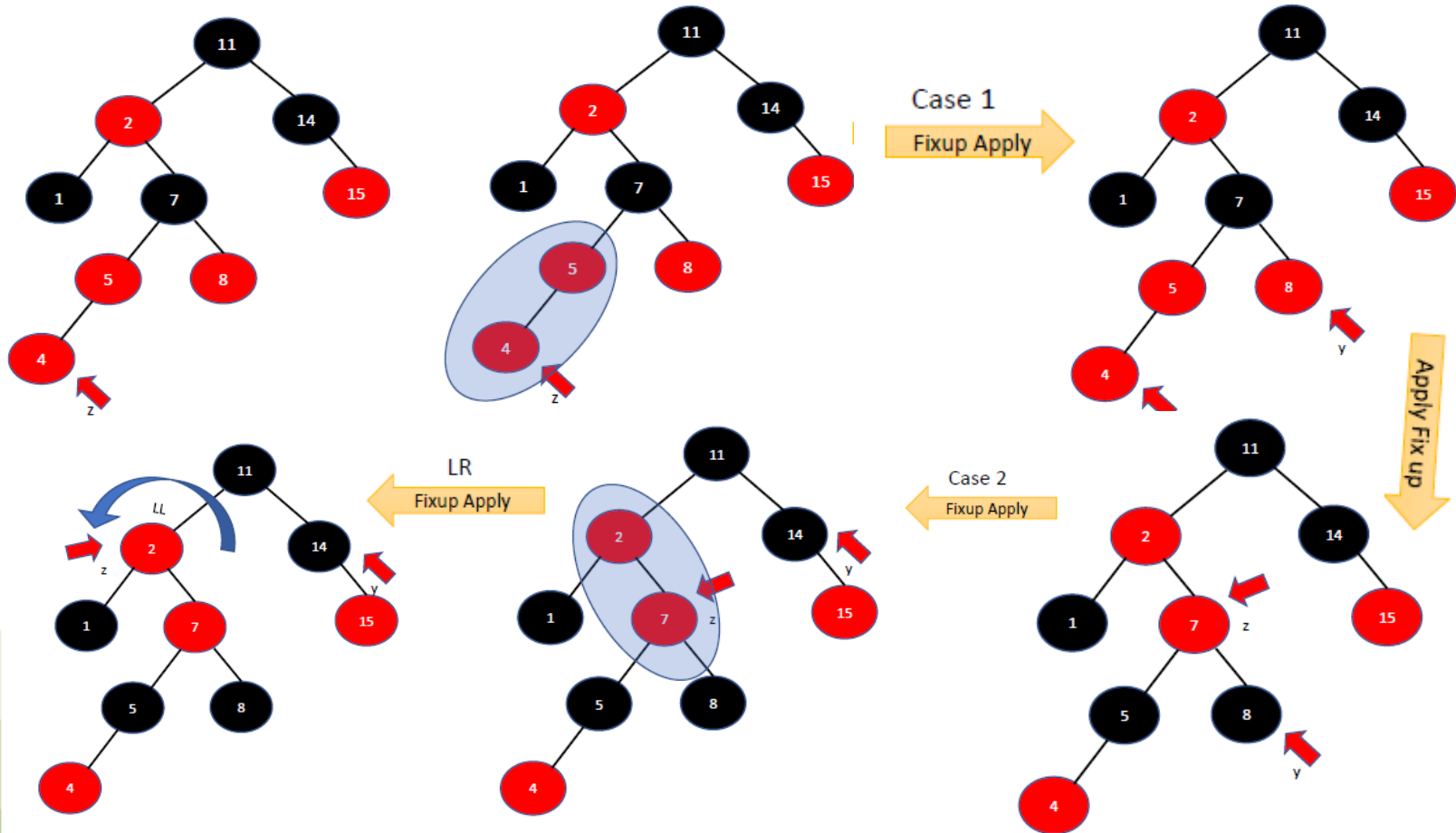
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -4



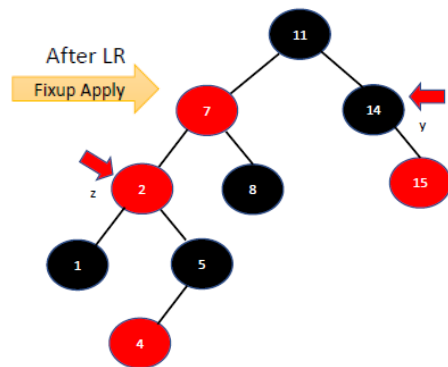
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -4



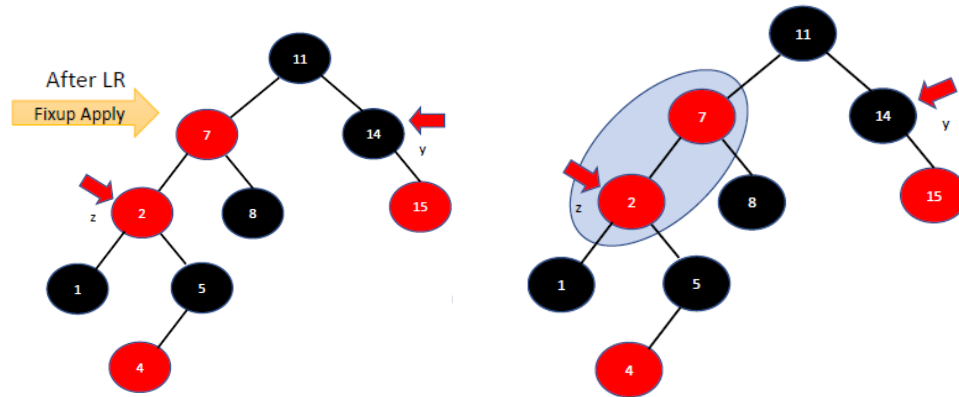
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -4



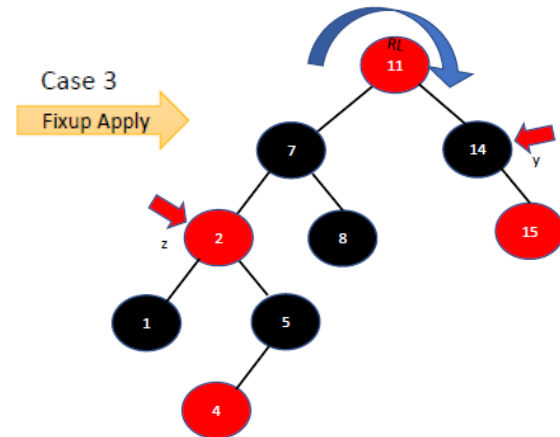
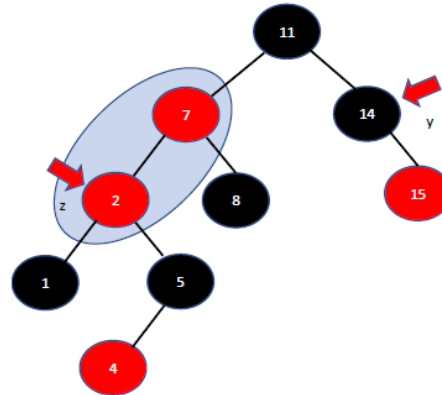
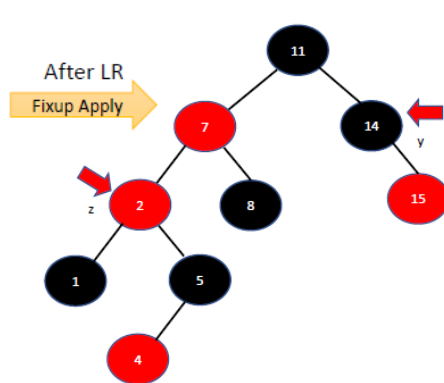
Red Black Tree

Insertion (Example 1):

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -4



Red Black Tree

Insertion:

```
RB-INSERT-FIXUP(T, z)
  while color[p[z]] = RED
    do if p[z] = left[p[p[z]]]
      then y ← right[p[p[z]]]
      if color[y] = RED
        then color[p[z]] ← BLACK
             color[y] ← BLACK
             color[p[p[z]]] ← RED
             z ← p[p[z]]
      else if z = right[p[p[z]]]
        then z ← p[z]
             LEFT-ROTATE(T, z)
             color[p[z]] ← BLACK
             color[p[p[z]]] ← RED
             RIGHT-ROTATE(T, p[p[z]])
    else (same as then clause with right and left exchanged)
  color[root(T)] ← BLACK
```

Once we apply Case 2 immediately, we apply Case 3

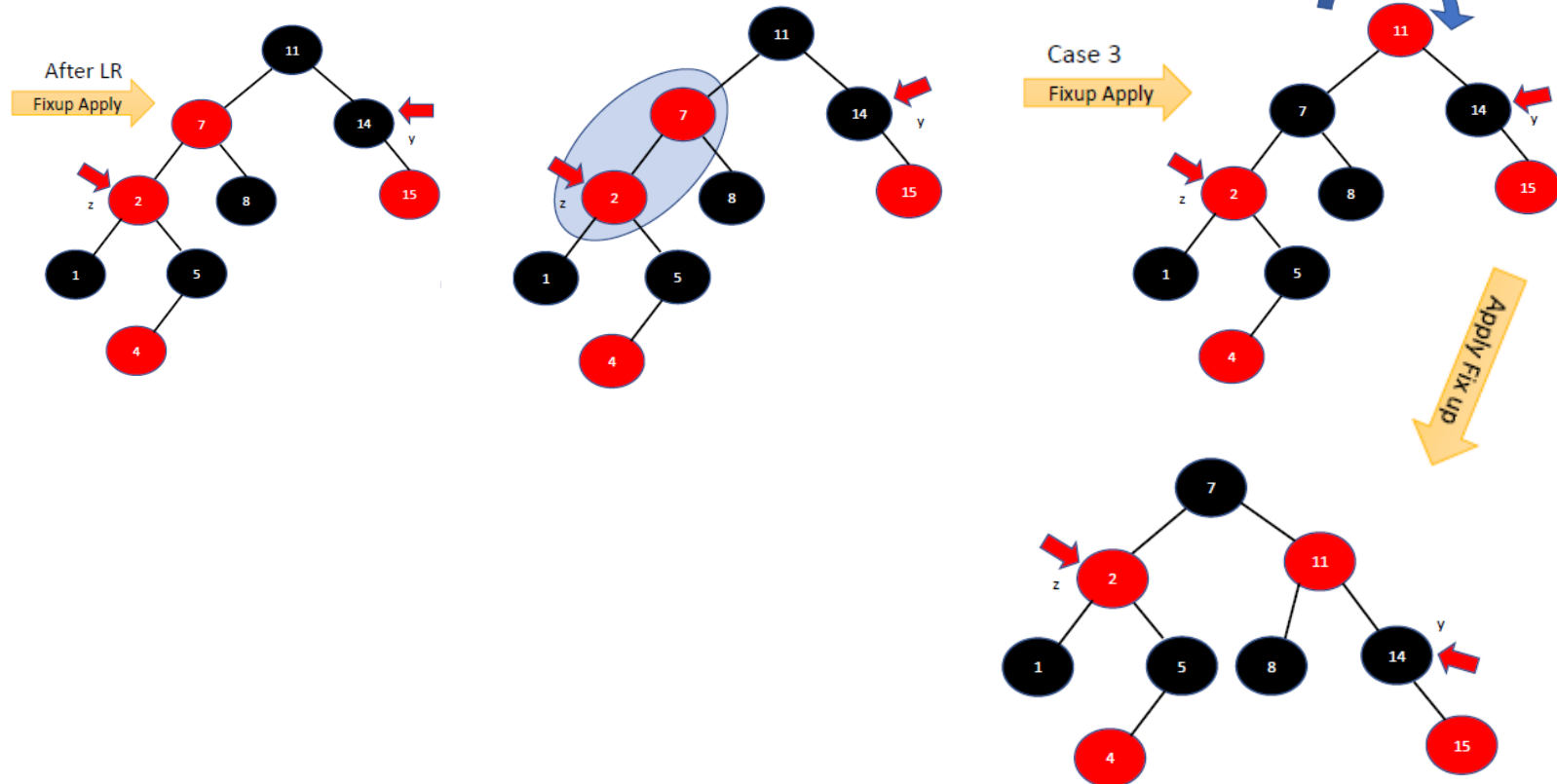
//Case 1
//Case 1
//Case 1
//Case 2
//Case 2
//Case 3
//Case 3
//Case 3

Insertion (Example 1): Red Black Tree

Insert the following elements into an empty RB-Tree.

[11, 2, 14, 1, 7, 15, 5, 8, 4]

Insert -4



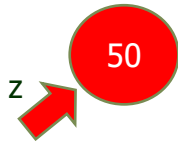
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -50



Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -50



Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -50



Red Black Tree

Insertion:

```
RB-INSERT-FIXUP(T, z)
while color[p[z]] = RED
do if p[z] = left[p[p[z]]]
then y ← right[p[p[z]]]
if color[y] = RED
then color[p[z]] ← BLACK
color[y] ← BLACK
color[p[p[z]]] ← RED
z ← p[p[z]]
else if z = right[p[p[z]]]
then z ← p[p[z]]
LEFT-ROTATE(T, z)
color[p[z]] ← BLACK
color[p[p[z]]] ← RED
RIGHT-ROTATE(T, p[p[z]])
else (same as then clause with right and left exchanged)
color[root(T)] ← BLACK
```

Once we apply Case 2 immediately, we apply Case 3

//Case 1
//Case 1
//Case 1
//Case 2
//Case 2
//Case 3
//Case 3
//Case 3

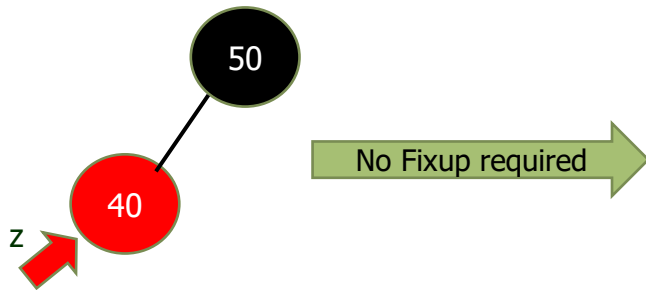
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -40



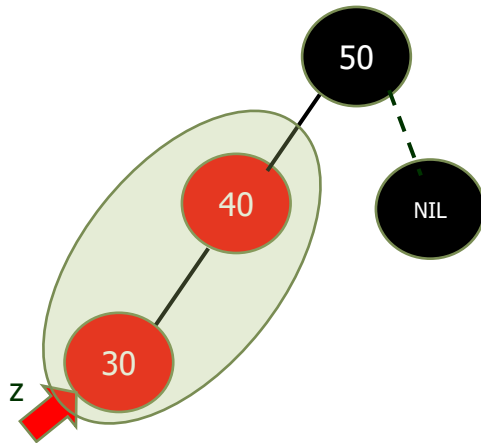
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -30



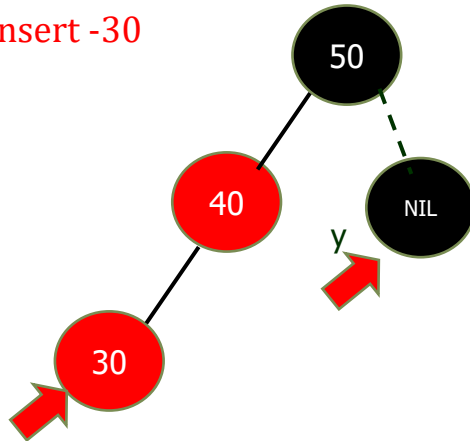
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

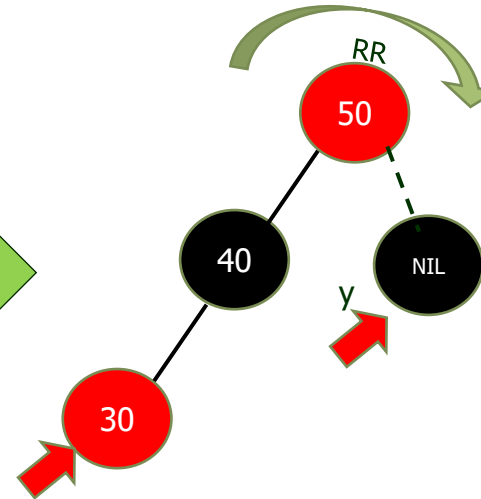
[50, 40, 30, 45, 20, 5]

Insert -30

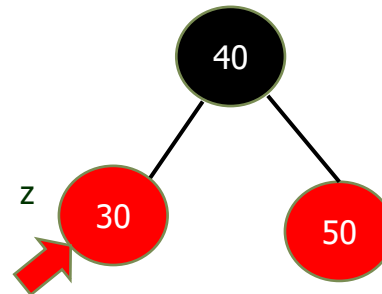


Case 3

Fixup Apply



After RR
Fixup Apply



Red Black Tree

Insertion:
RB-INSERT-FIXUP(T, z)
while color[p[z]] = RED
do if p[z] = left[p[p[z]]]
then z ← right[p[p[z]]]
if color[z] = RED
then color[p[z]] ← BLACK
color[z] ← BLACK
color[p[p[z]]] ← RED
z ← p[p[z]]
else if z = right[p[p[z]]]
then z ← p[p[z]]
LEFT-ROTATE(T, z)
color[p[z]] ← BLACK
color[p[p[z]]] ← RED
RIGHT-ROTATE(T, p[p[z]])
else (same as then clause with right and left exchanged)
color[p[p[z]]] ← BLACK

Once we apply Case 2 immediately, we apply Case 3

Case 1
Case 1
Case 1
Case 2
Case 3
Case 3

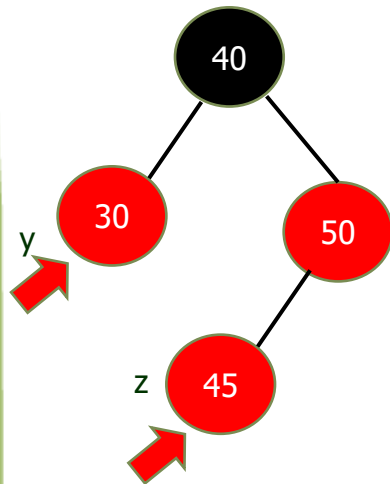
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -45



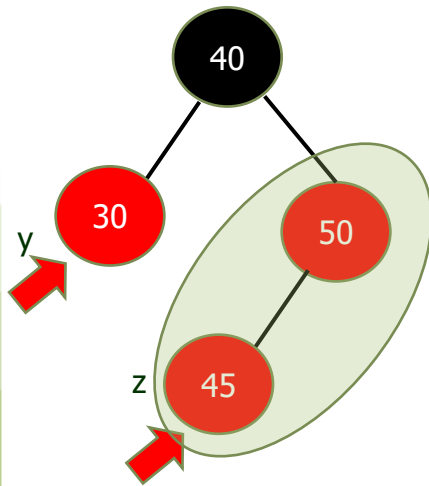
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -45



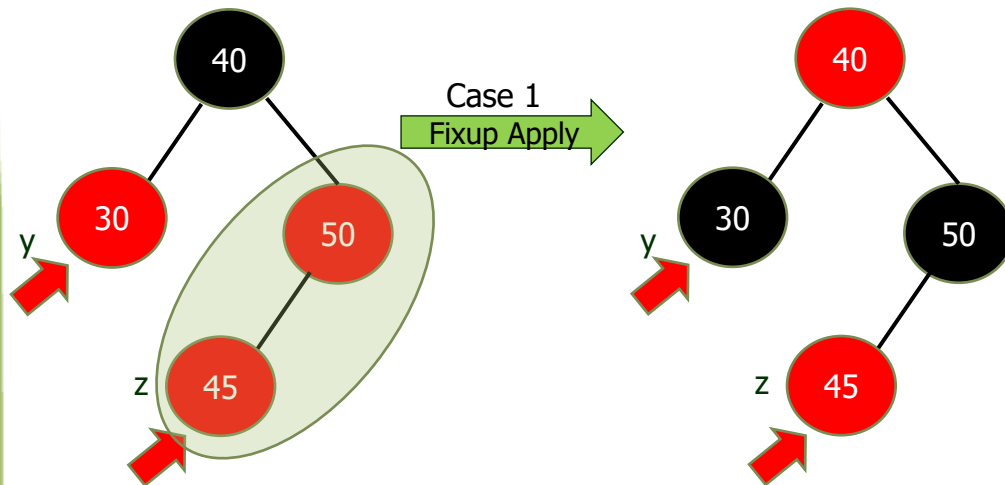
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -45



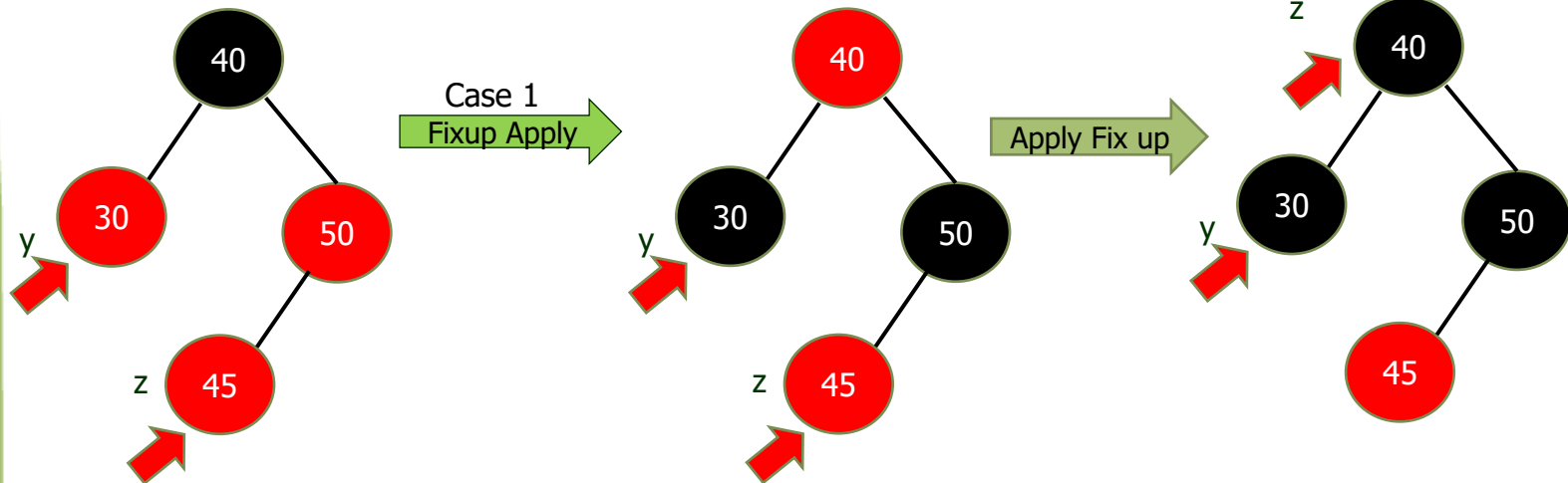
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -45



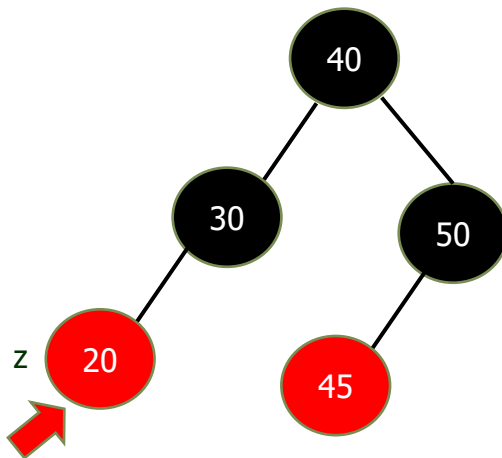
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -20



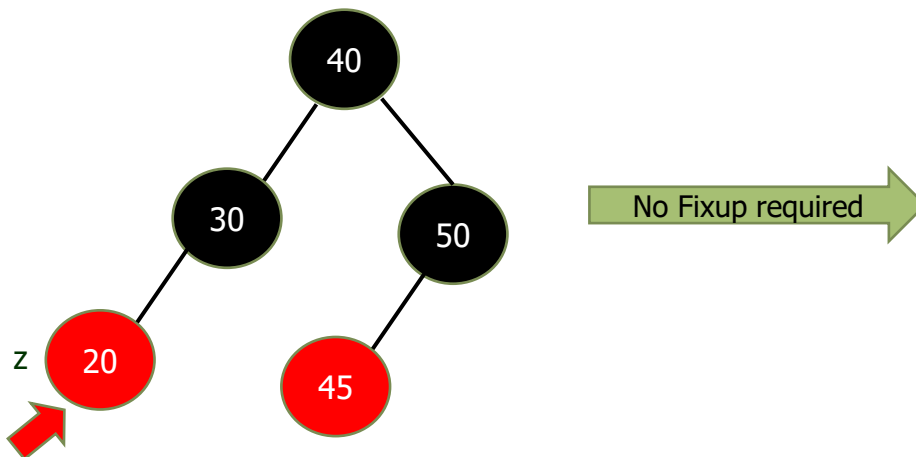
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -20



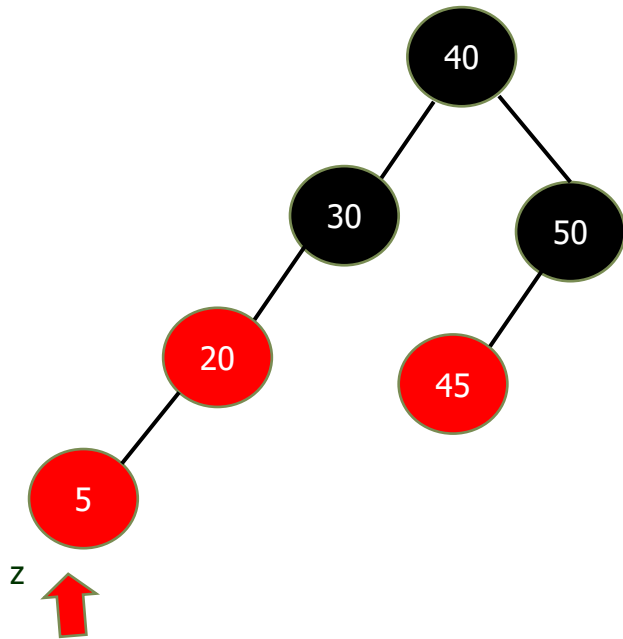
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -5



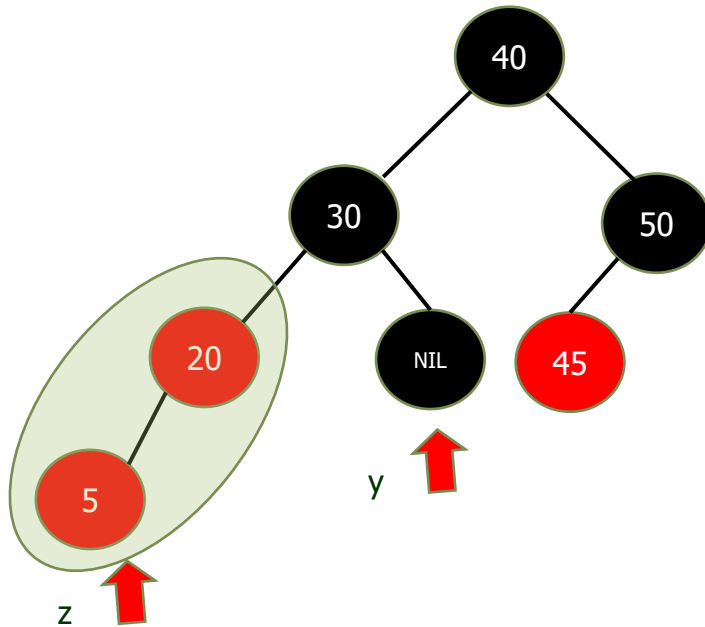
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -5



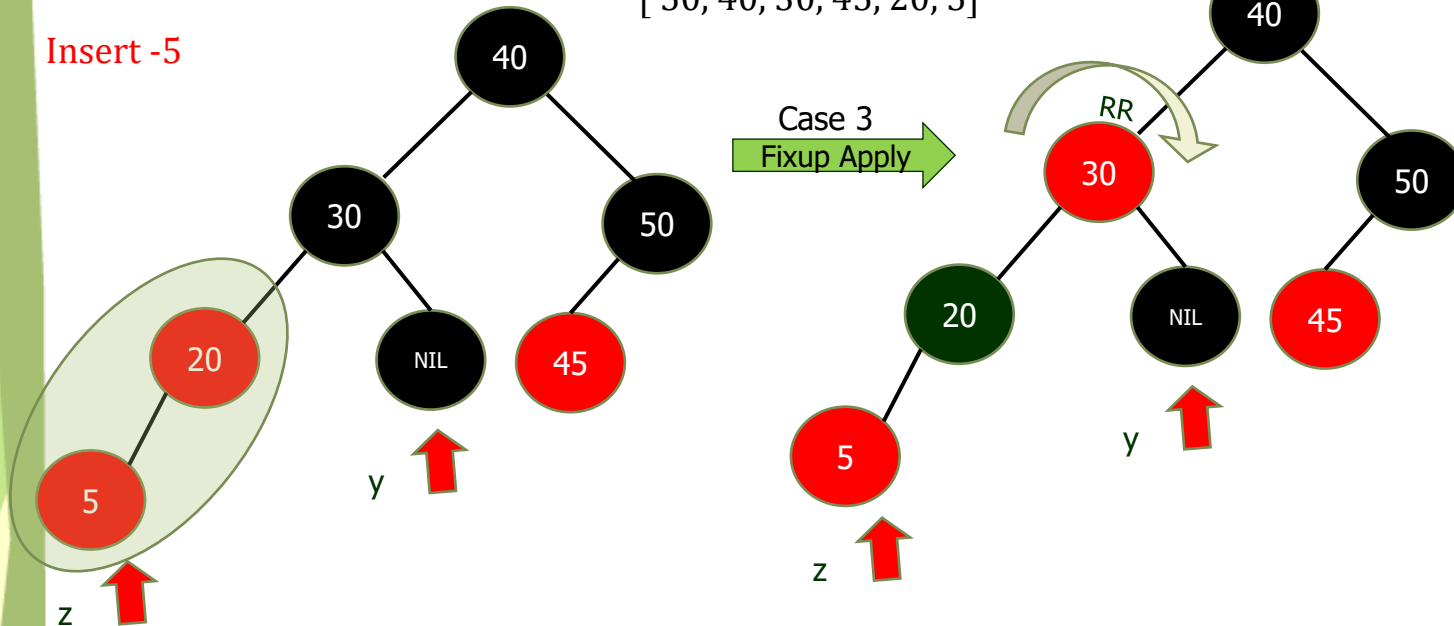
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -5



Red Black Tree

Insertion:

```

RB-INSERT-FIXUP(T, z)
while color[p[z]] = RED
do if p[z] = left[p[p[z]]]
then y ← right[p[p[z]]]
if color[y] = RED
then color[p[z]] ← BLACK
color[y] ← BLACK
color[p[p[z]]] ← RED
z ← p[p[z]]
else if z = right[p[p[z]]]
then z ← p[z]
LEFT-ROTATE(T, z)
color[p[z]] ← BLACK
color[p[p[z]]] ← RED
RIGHT-ROTATE(T, p[p[z]])
else (same as then clause with right and left exchanged)
color[root(T)] ← BLACK
    
```

Once we apply Case 2 immediately, we apply Case 3

//Case 1
//Case 1
//Case 1
//Case 1
//Case 2
//Case 2
//Case 3
//Case 3

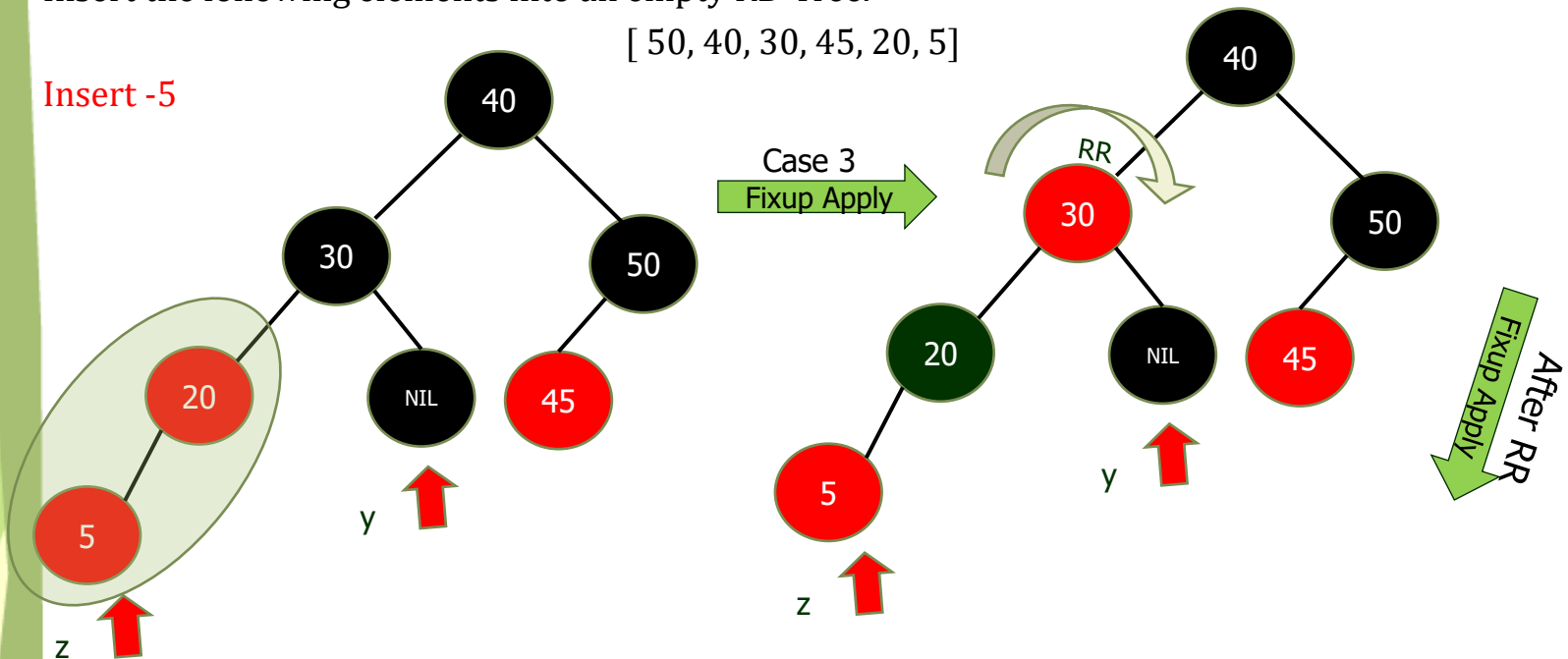
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -5



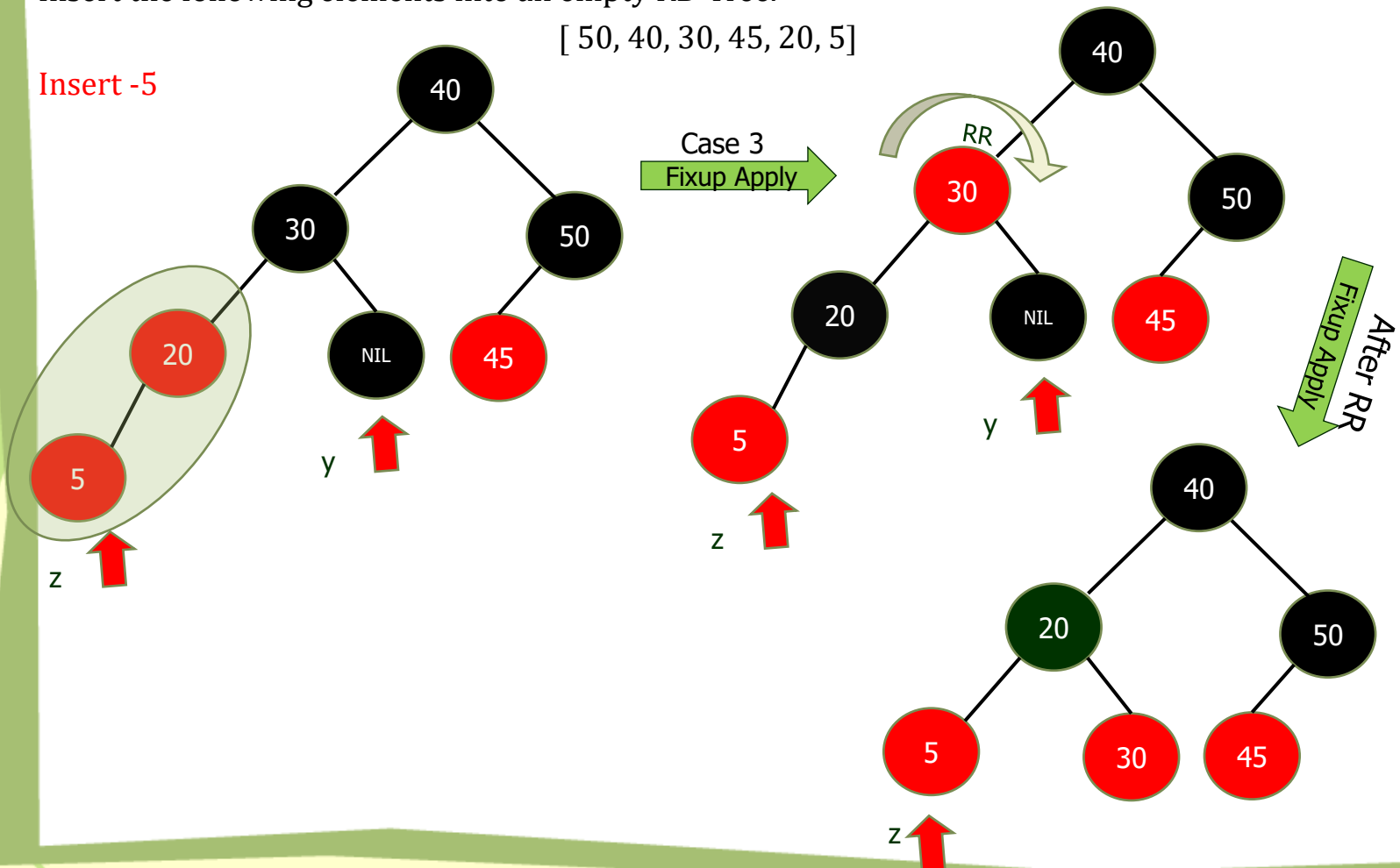
Red Black Tree

Insertion (Example 2):

Insert the following elements into an empty RB-Tree.

[50, 40, 30, 45, 20, 5]

Insert -5



Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -41



Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -50



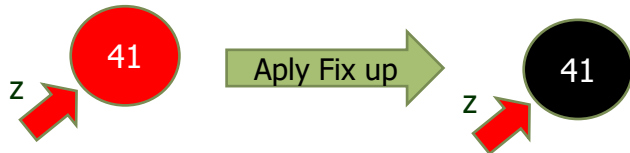
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -50



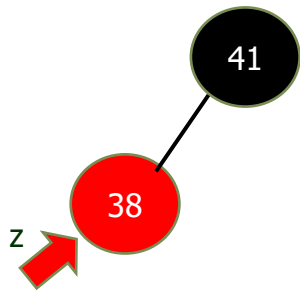
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -38



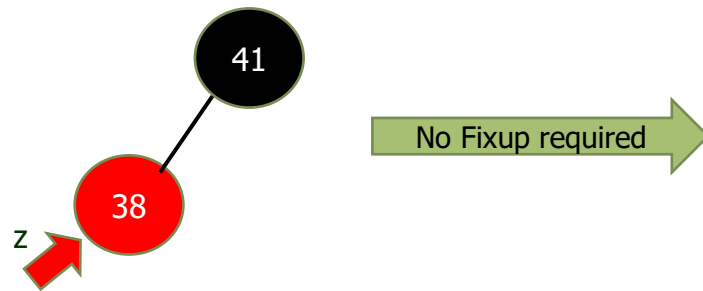
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -38



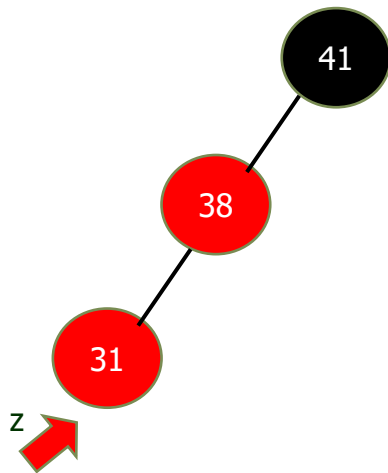
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -31



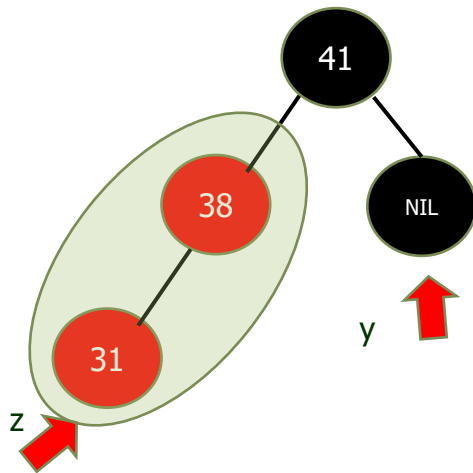
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -31



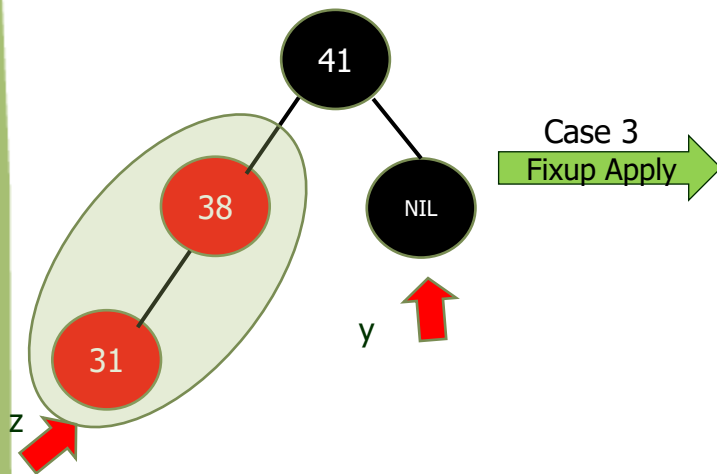
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -31



Red Black Tree

Insertion:

```

RB-INSERT-FIXUP(T, z)
while color[p[z]] = RED
do if p[z] = left[p[p[z]]]
then y ← right[p[p[z]]]
if color[y] = RED
then color[p[z]] ← BLACK
color[y] ← BLACK
color[p[p[z]]] ← RED
z ← p[p[z]]
else if z = right[p[p[z]]]
then z ← p[p[z]]
LEFT-ROTATE(T, z)
color[p[z]] ← BLACK
color[p[p[z]]] ← RED
RIGHT-ROTATE(T, p[p[z]])
else (same as then clause with right, and left, exchanged)
color[root[T]] ← BLACK
    
```

Once we apply Case 2 immediately, we apply Case 3

//Case 1
//Case 1
//Case 1
//Case 2
//Case 2
//Case 3
//Case 3
//Case 3

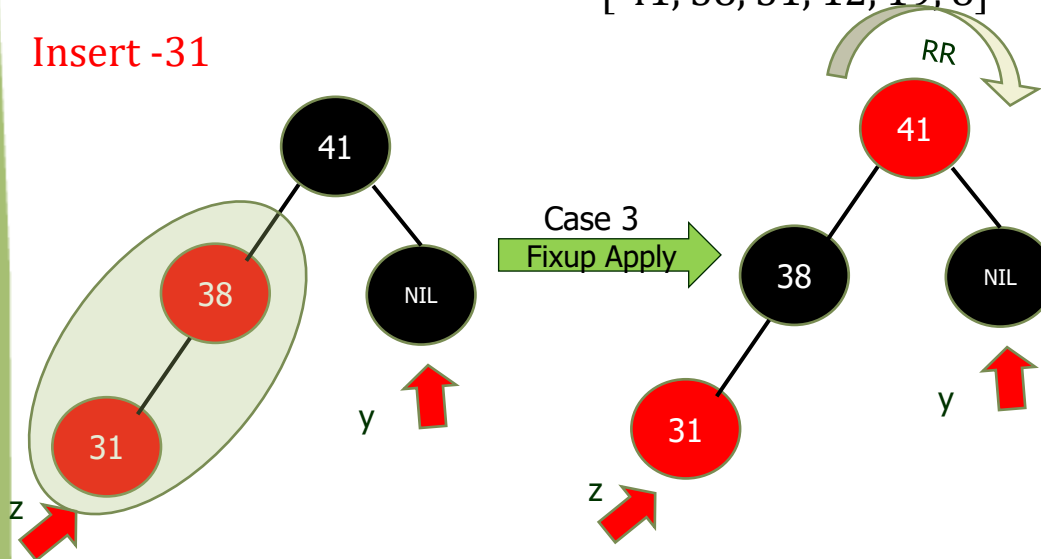
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -31



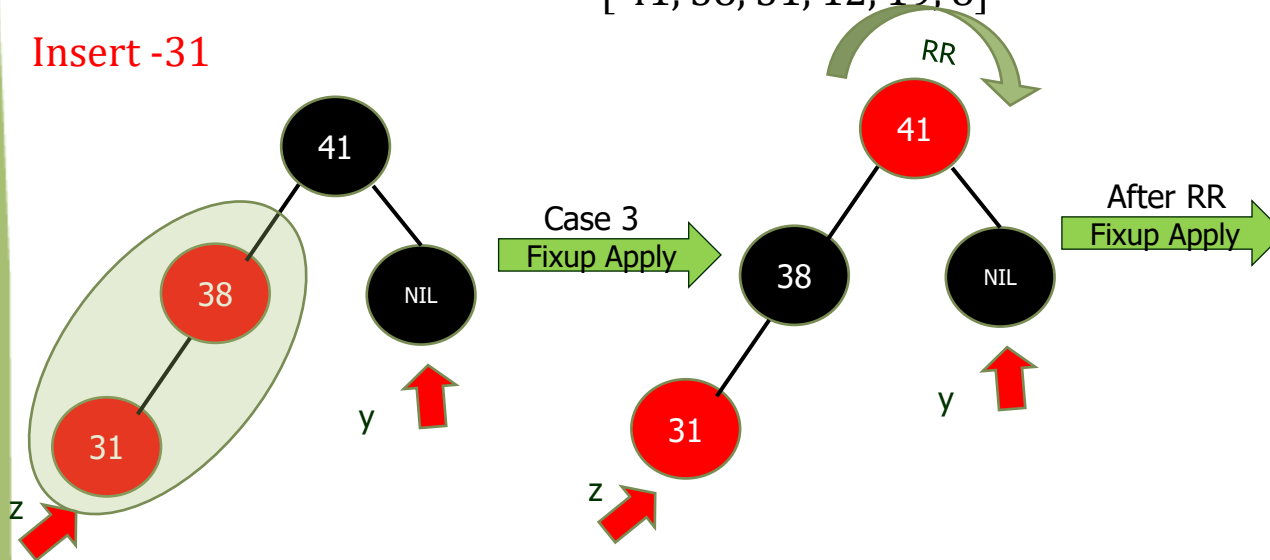
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -31



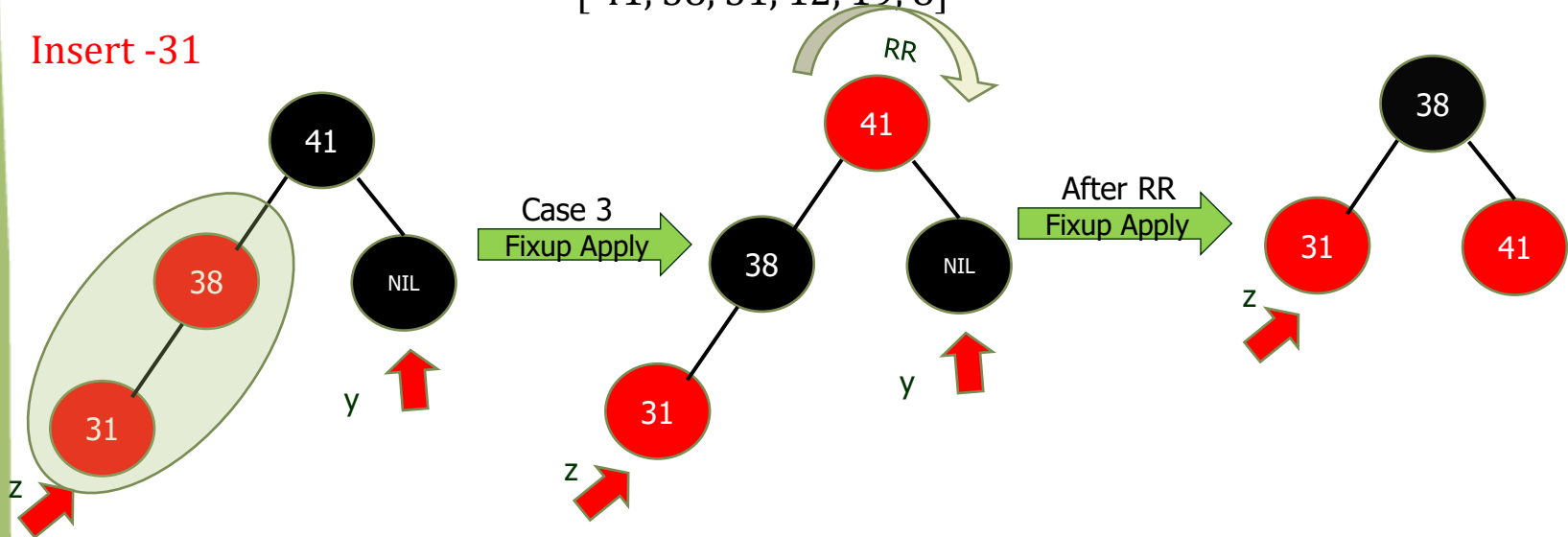
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -31



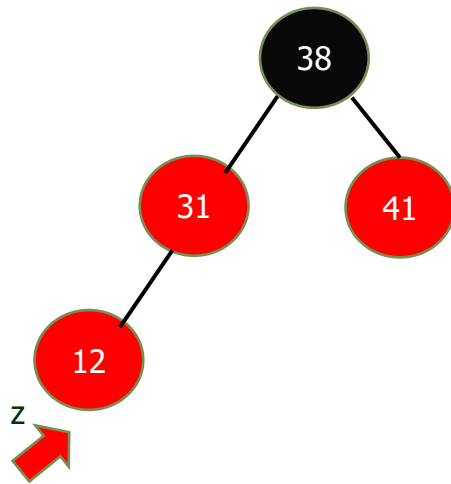
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -12



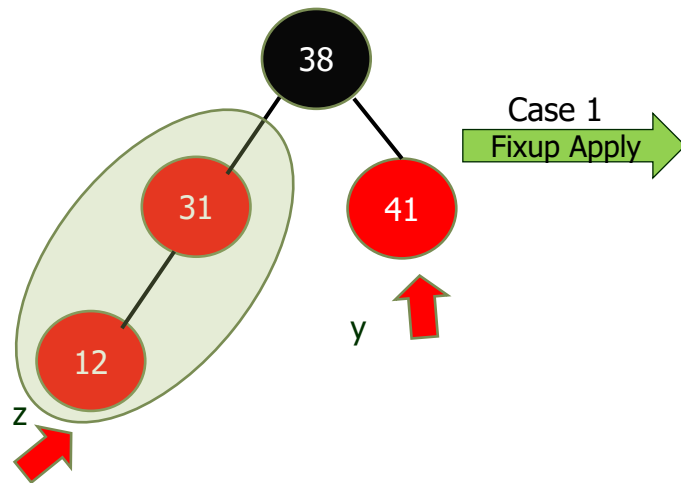
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -12



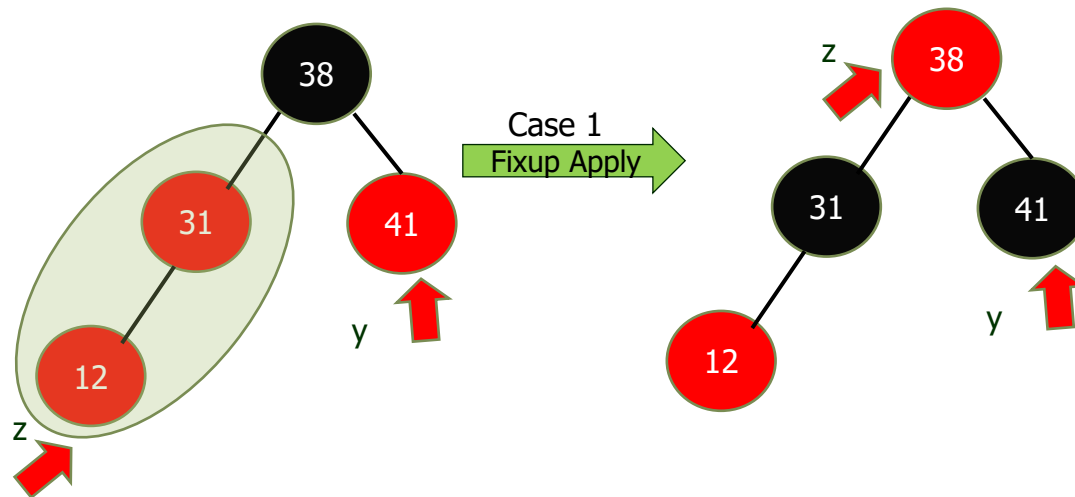
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -12



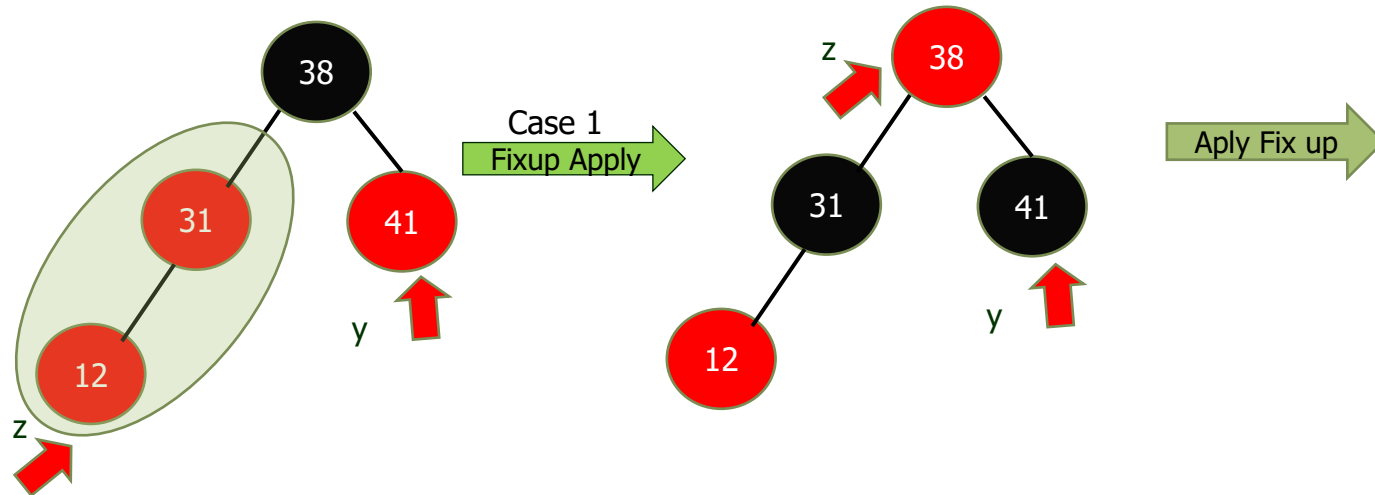
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -12



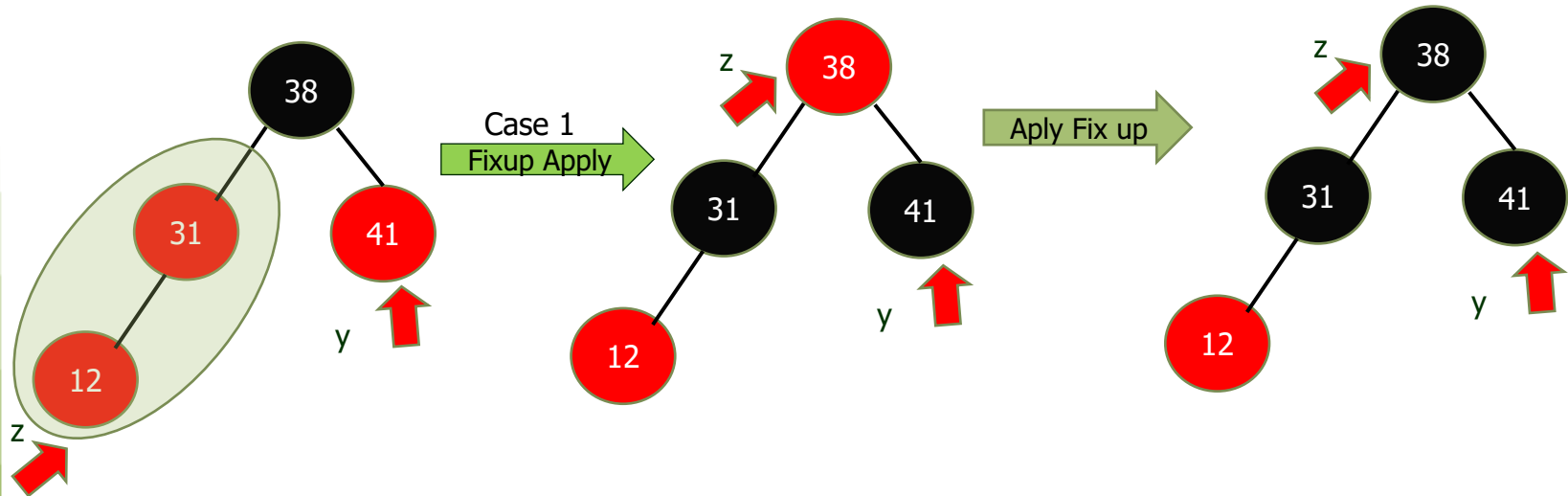
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -12



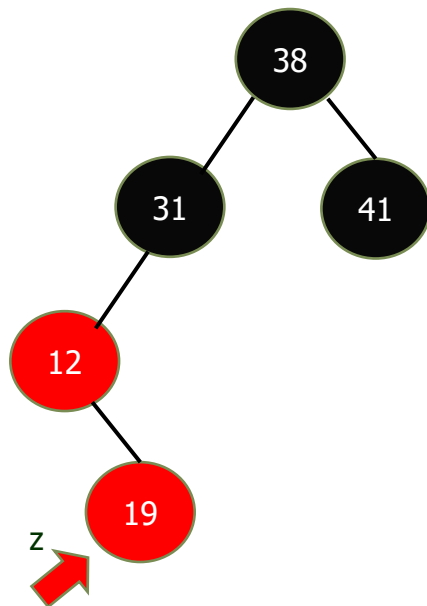
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -19



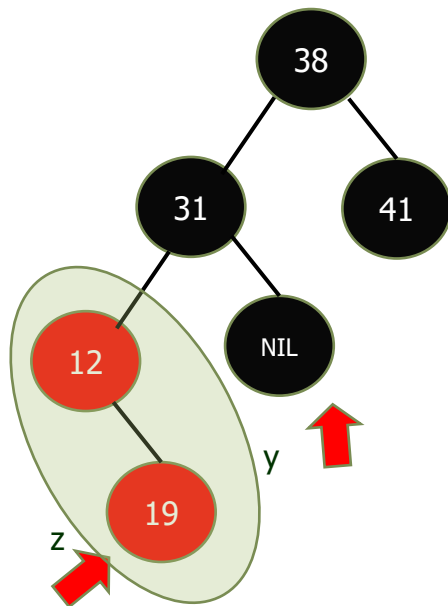
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -19



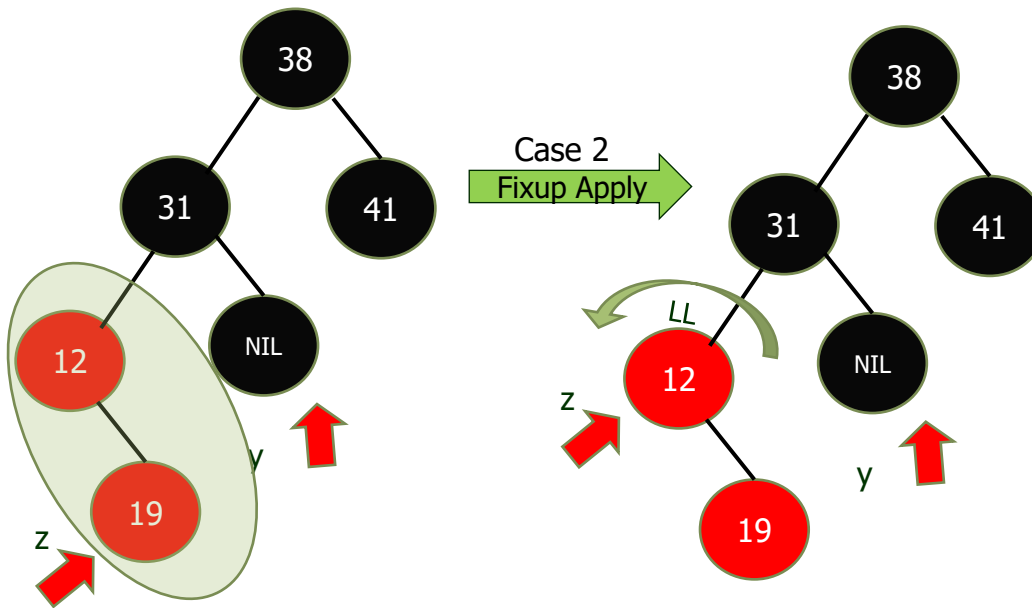
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -19



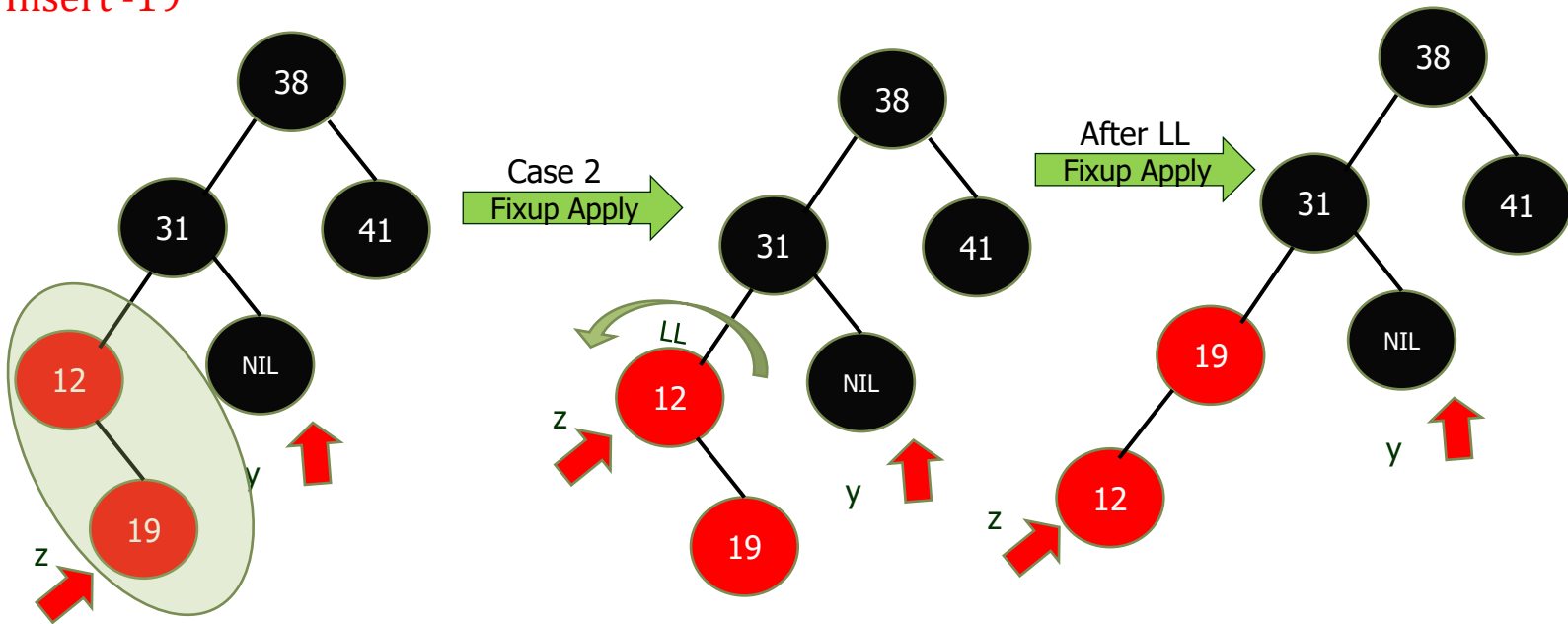
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -19



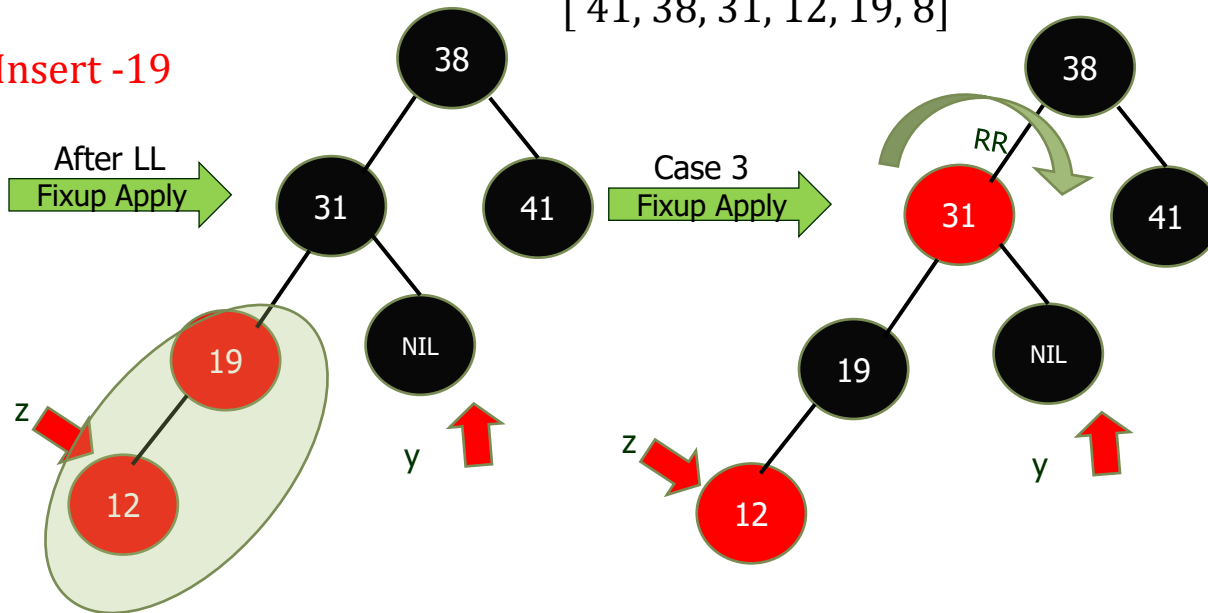
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -19



Red Black Tree

Insertion:

```

RB-INSERT-FIXUP(T, z)
while color[p[z]] = RED
do if p[z] = left[p[p[z]]]
then y ← right[p[p[z]]]
if color[y] = RED
then color[p[z]] ← BLACK
color[y] ← BLACK
color[p[p[z]]] ← RED
z ← p[p[z]]
else if z = right[p[p[z]]]
then z ← p[z]
LEFT-ROTATE(T, z)
color[p[z]] ← BLACK
color[p[p[z]]] ← RED
RIGHT-ROTATE(T, p[p[z]])
else (same as then clause with right and left exchanged)
color[root(T)] ← BLACK
    
```

Once we apply Case 2 immediately, we apply Case 3

//Case 1
//Case 1
//Case 1
//Case 2
//Case 2
//Case 3
//Case 3
//Case 3

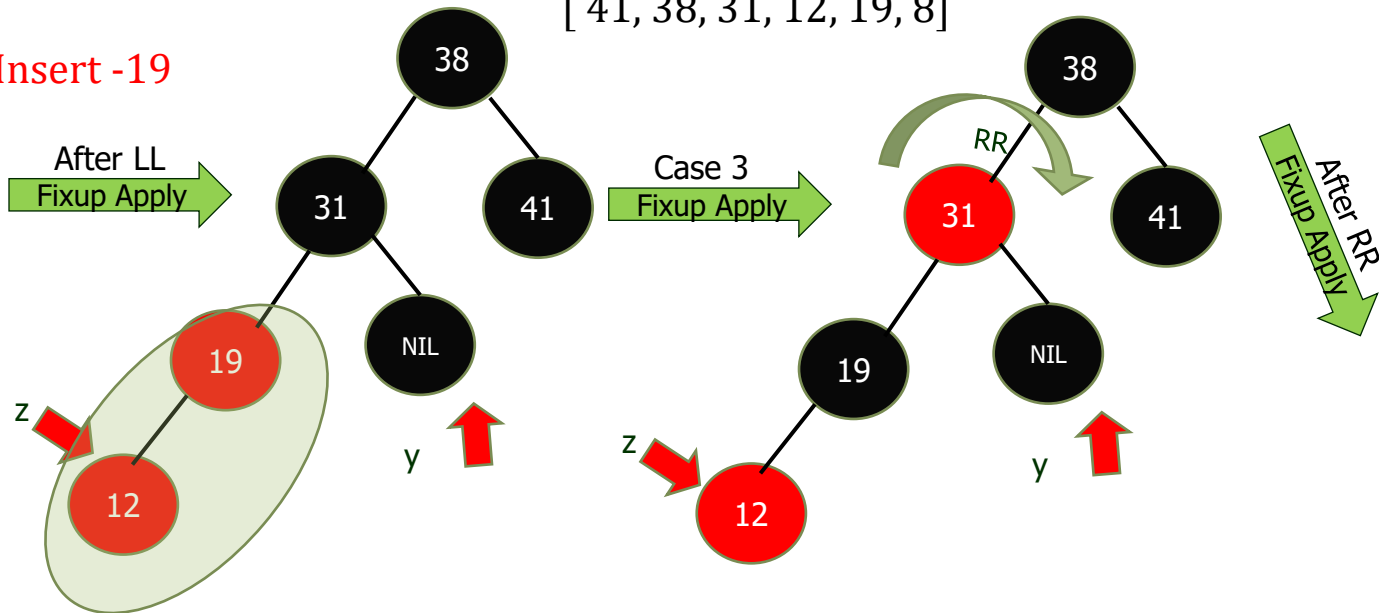
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -19



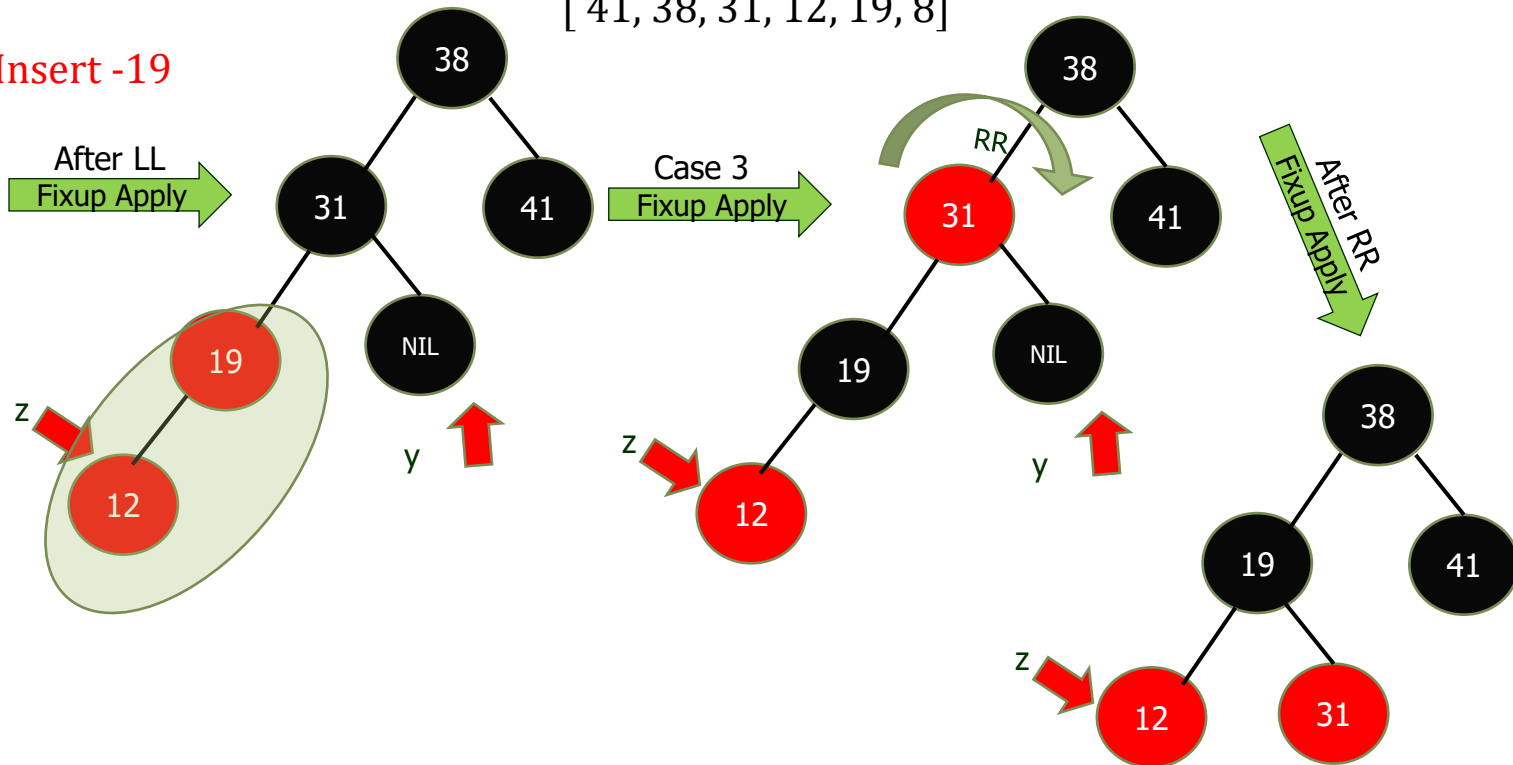
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -19



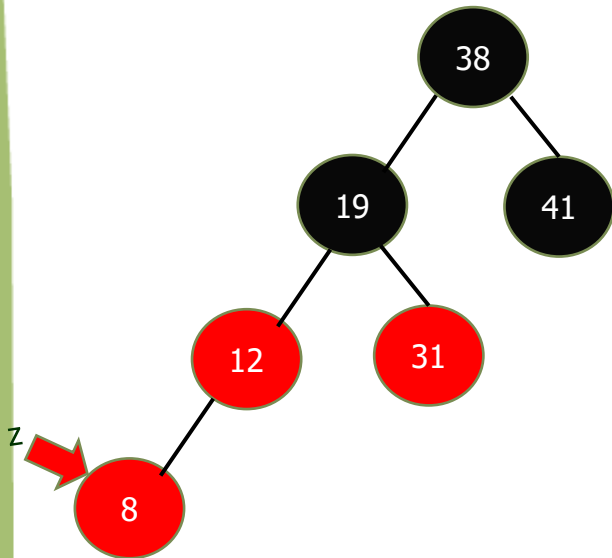
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -8



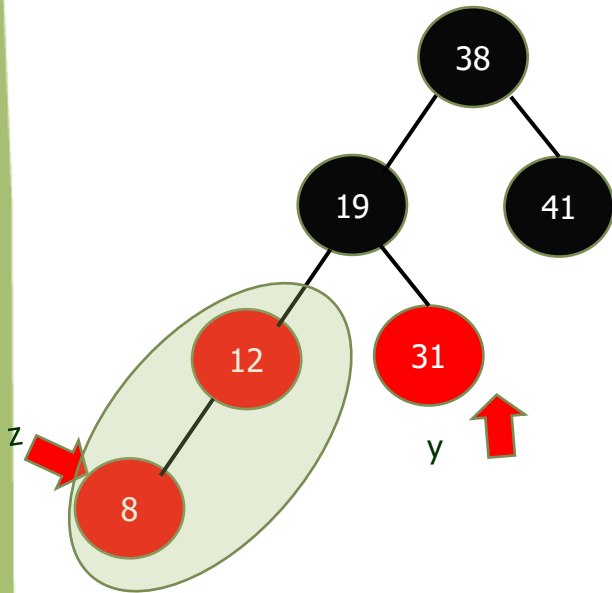
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -8



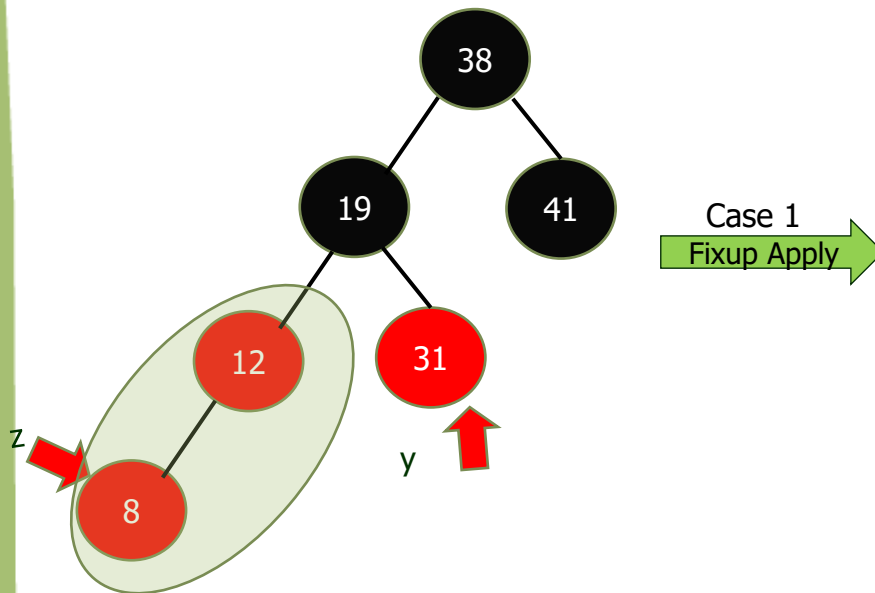
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -8



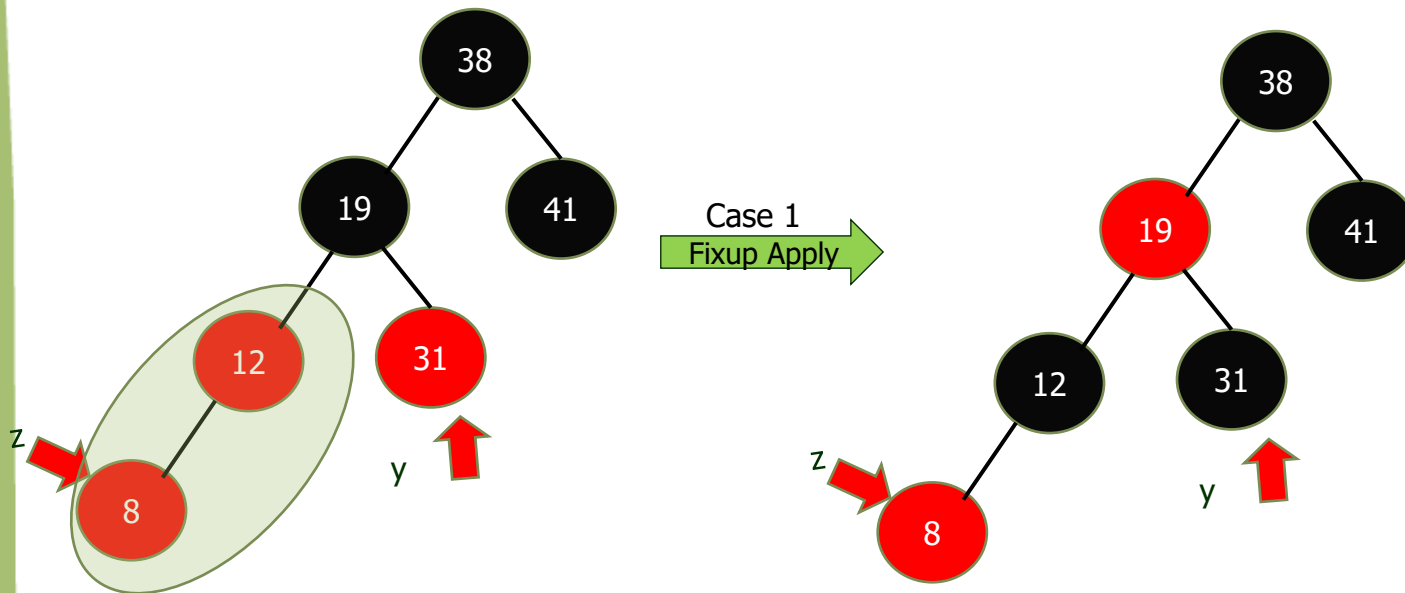
Red Black Tree

Insertion (Example 3):

Insert the following elements into an empty RB-Tree.

[41, 38, 31, 12, 19, 8]

Insert -8



Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10 ,15 , 25, 20 ,30]

Insert -5



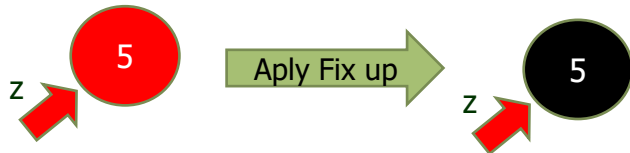
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -5



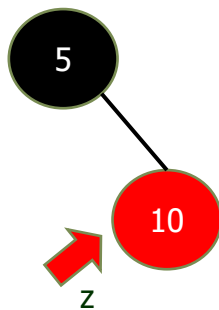
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10 ,15 , 25, 20 ,30]

Insert -10



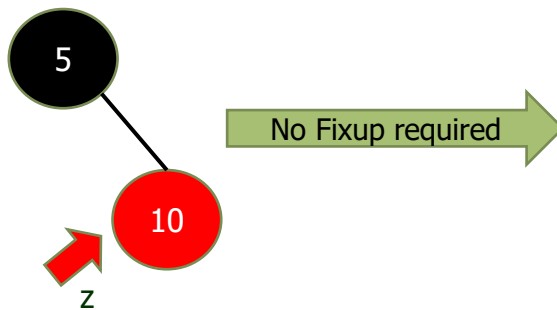
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -10



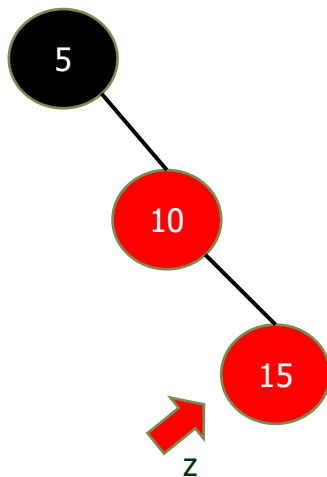
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -15



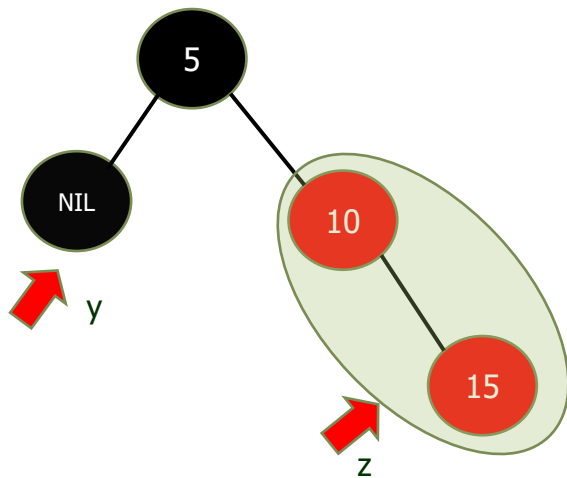
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -15



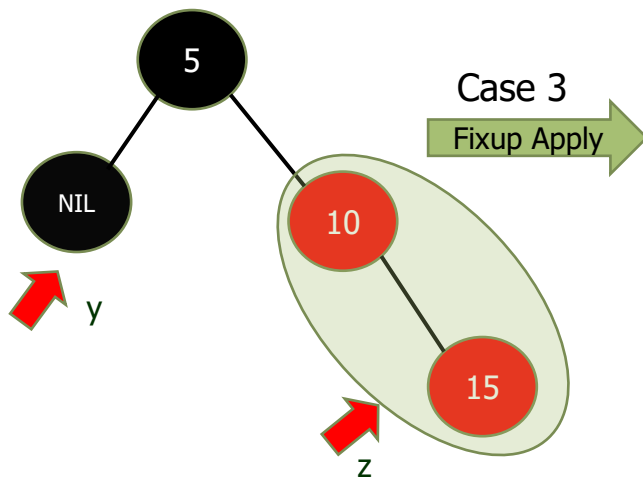
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -15



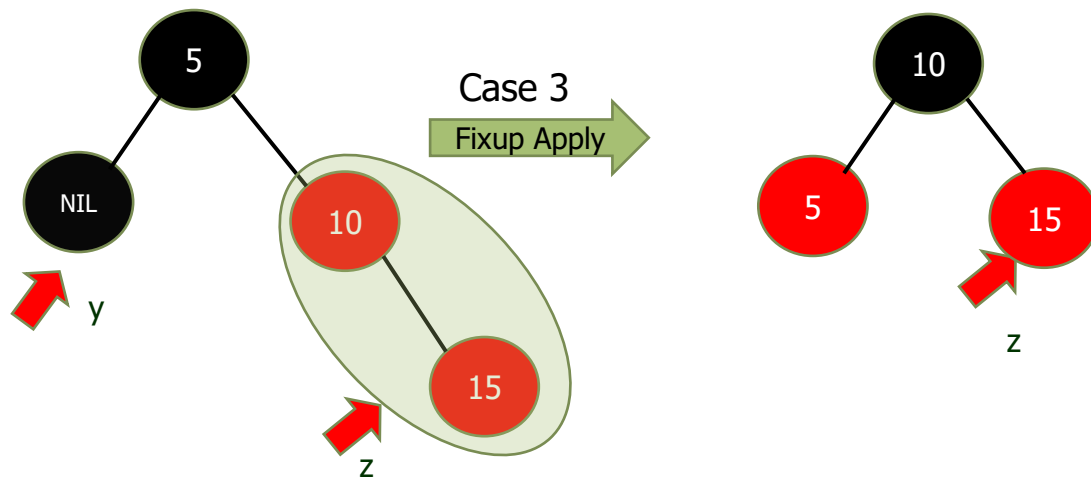
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -15



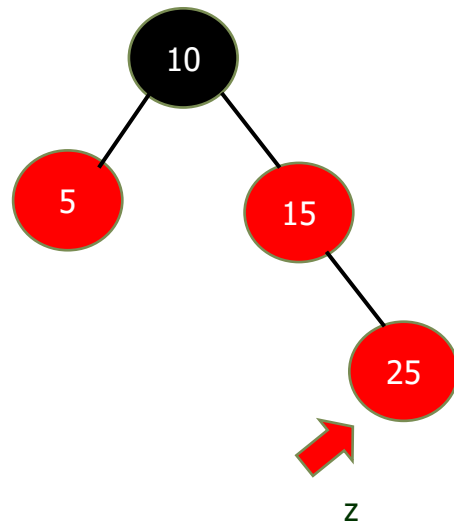
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -25



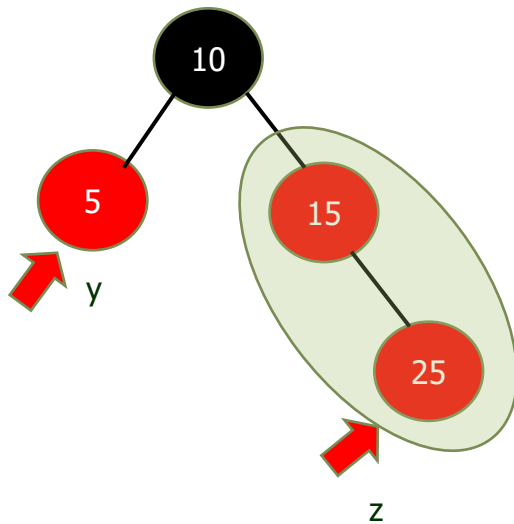
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -25



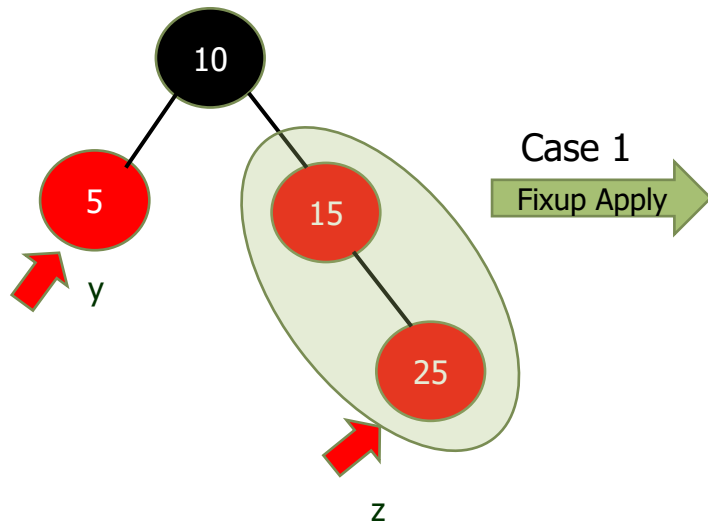
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -25



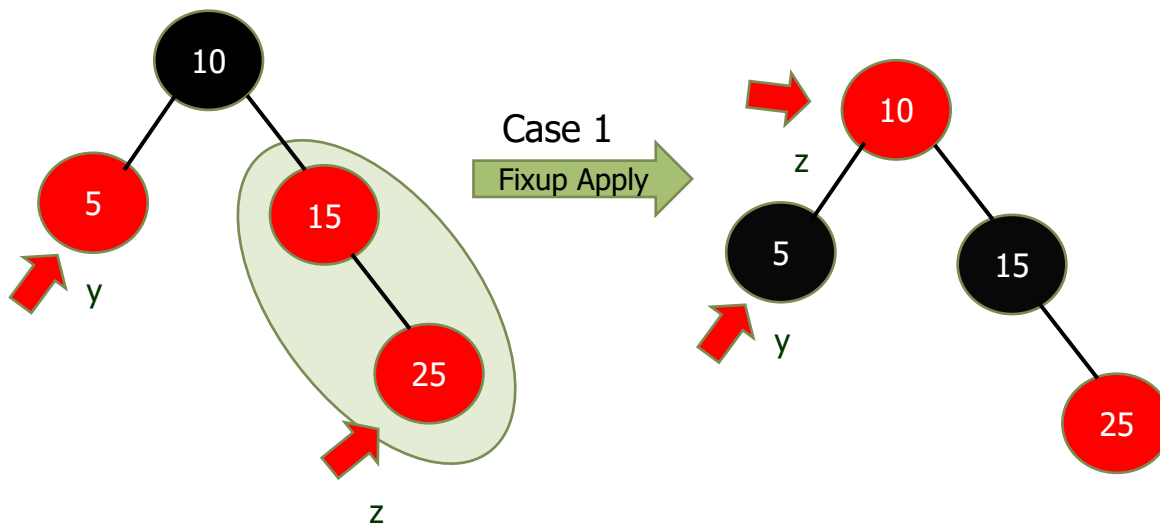
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -25



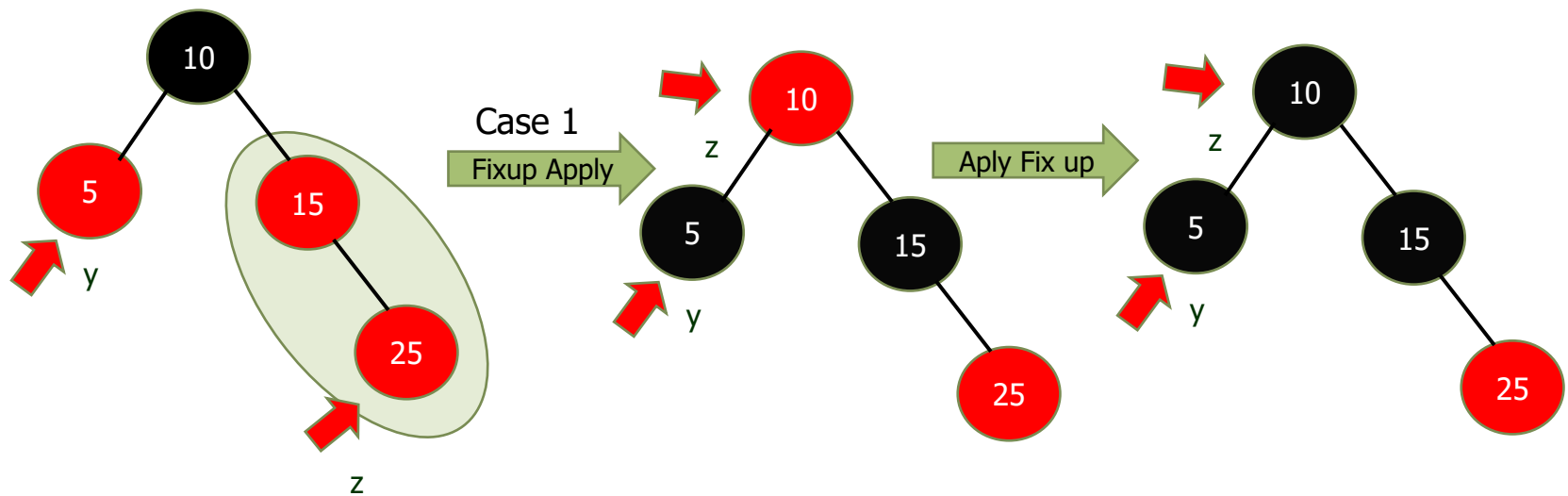
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -25



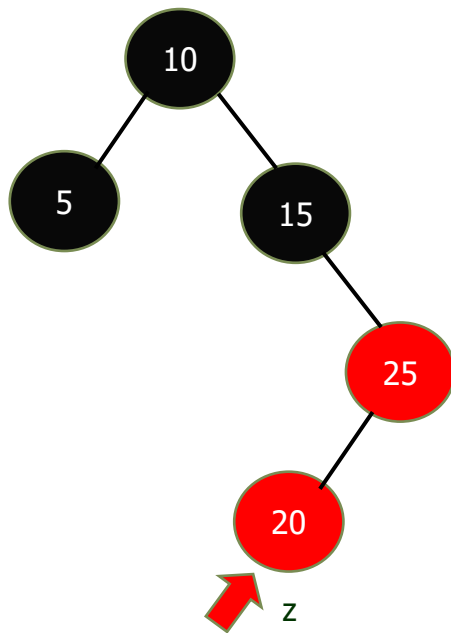
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -20



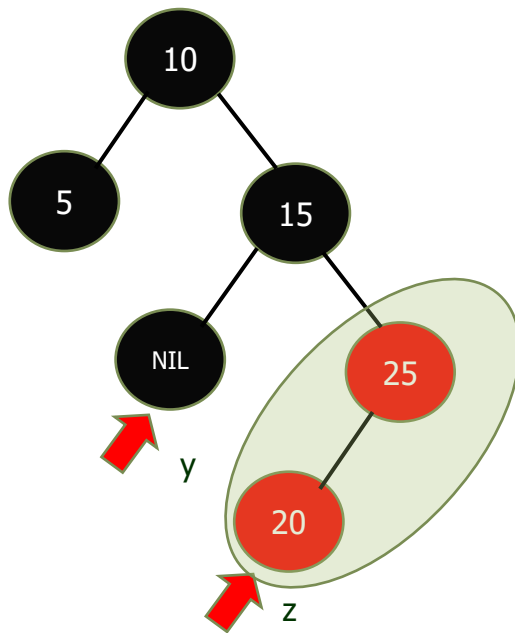
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -20



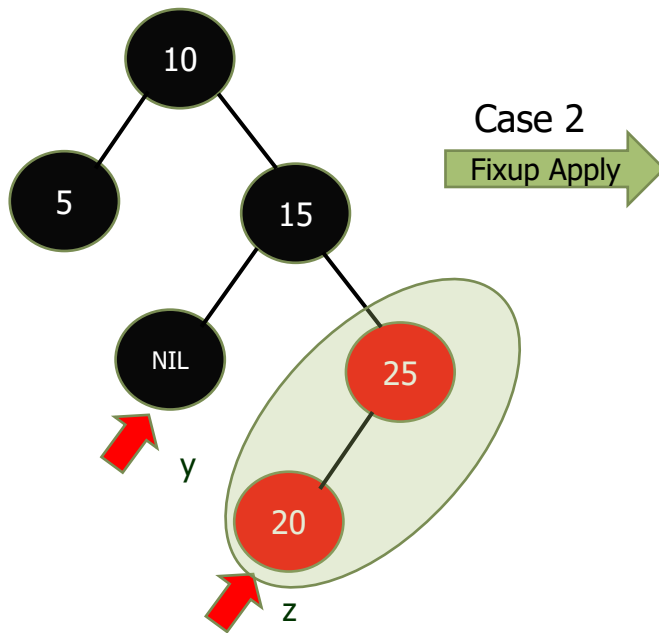
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -20



Red Black Tree

Insertion:

```

RB-INSERT-FIXUP(T, z)
while color[p[z]] = RED
do if p[z] = left[p[p[z]]]
then y ← right[p[p[z]]]
if color[y] = RED
then color[p[z]] ← BLACK
color[y] ← BLACK
color[p[p[z]]] ← RED
z ← p[p[z]]
else if z = right[p[p[z]]]
then z ← p[p[z]]
LEFT-ROTATE(T, z)
color[p[z]] ← BLACK
color[p[p[z]]] ← RED
RIGHT-ROTATE(T, p[p[z]])
else (same as then clause with right and left exchanged)
color[root(T)] ← BLACK
    
```

Once we apply
Case 2
immediately, we
apply Case 3

//Case 1
//Case 1
//Case 1
//Case 2
//Case 2
//Case 3
//Case 3
//Case 3

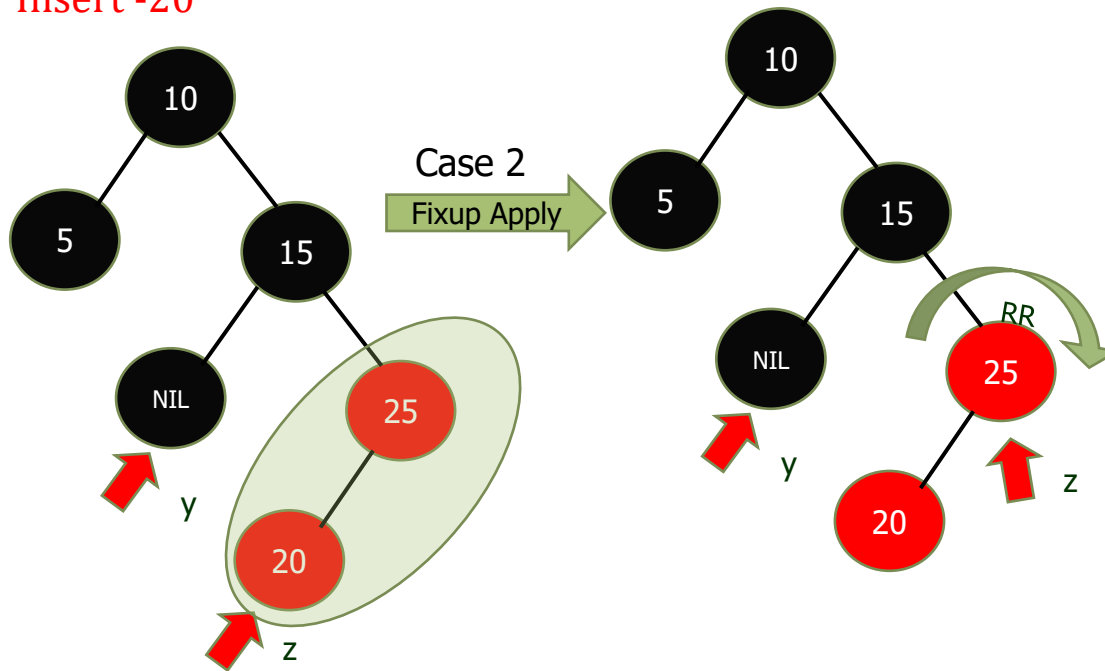
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -20



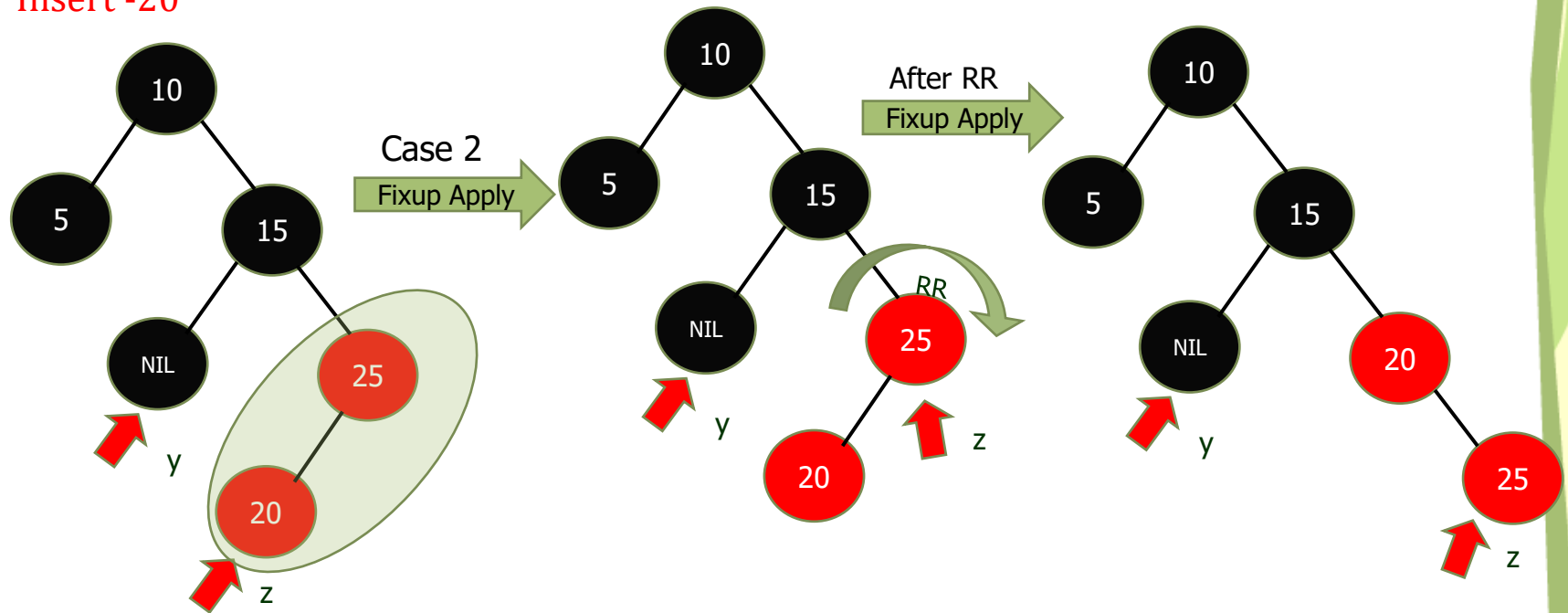
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -20



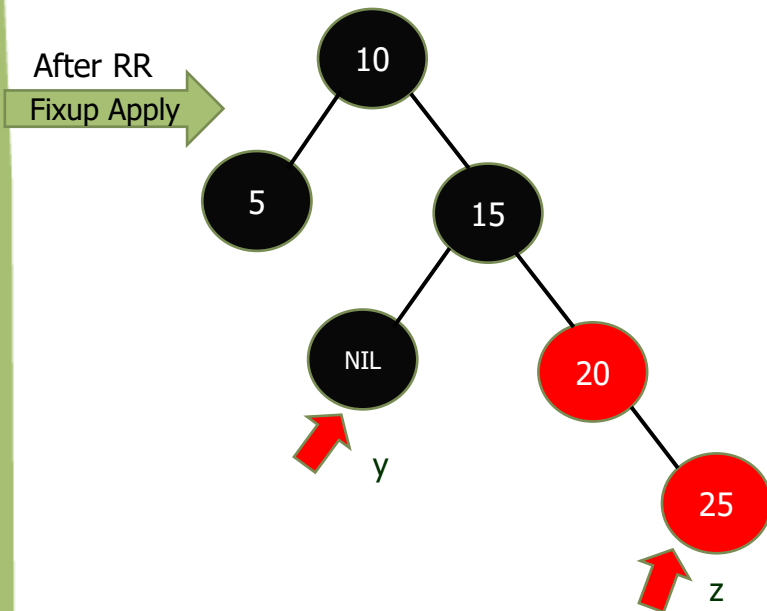
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -20



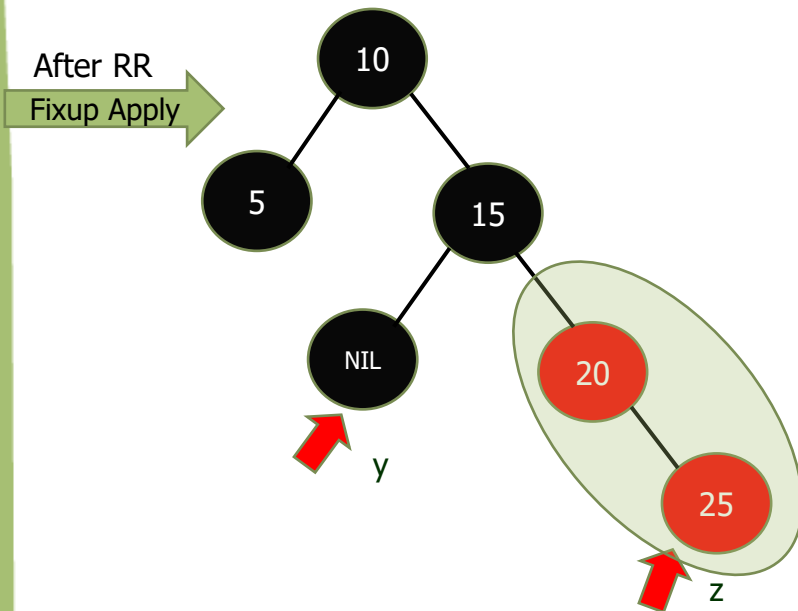
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -20



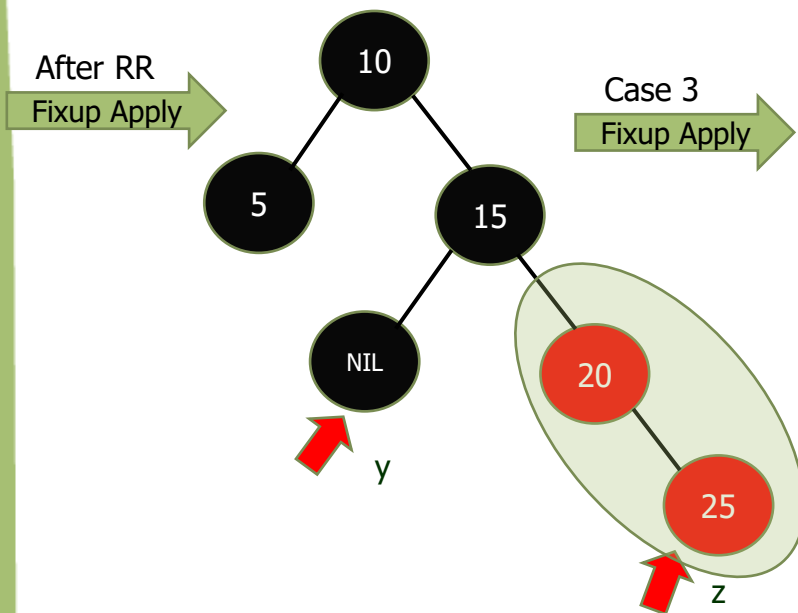
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -20



Red Black Tree

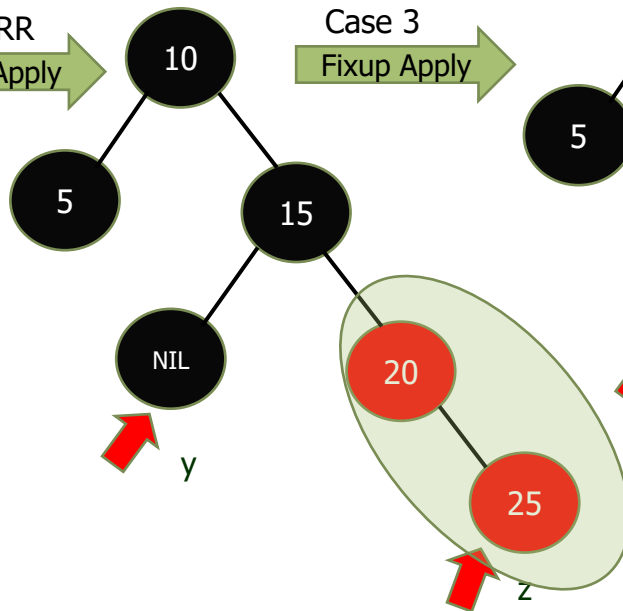
Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

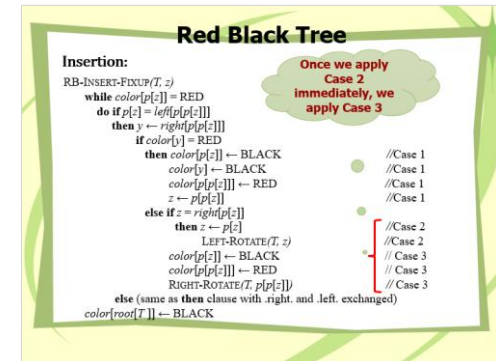
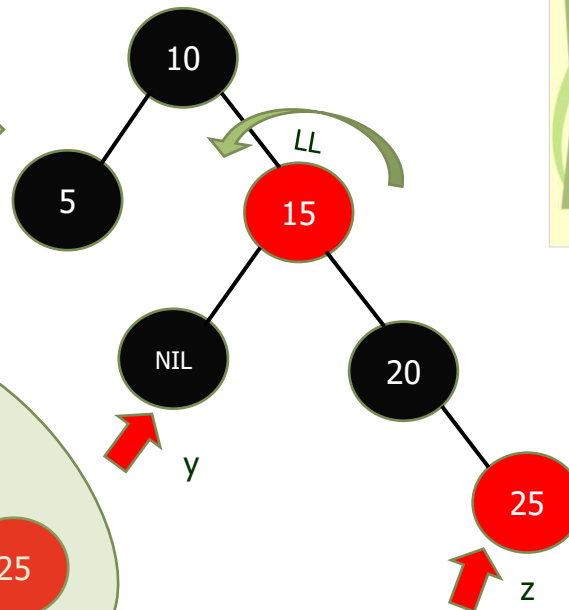
Insert -20

After RR
Fixup Apply



Case 3

Fixup Apply



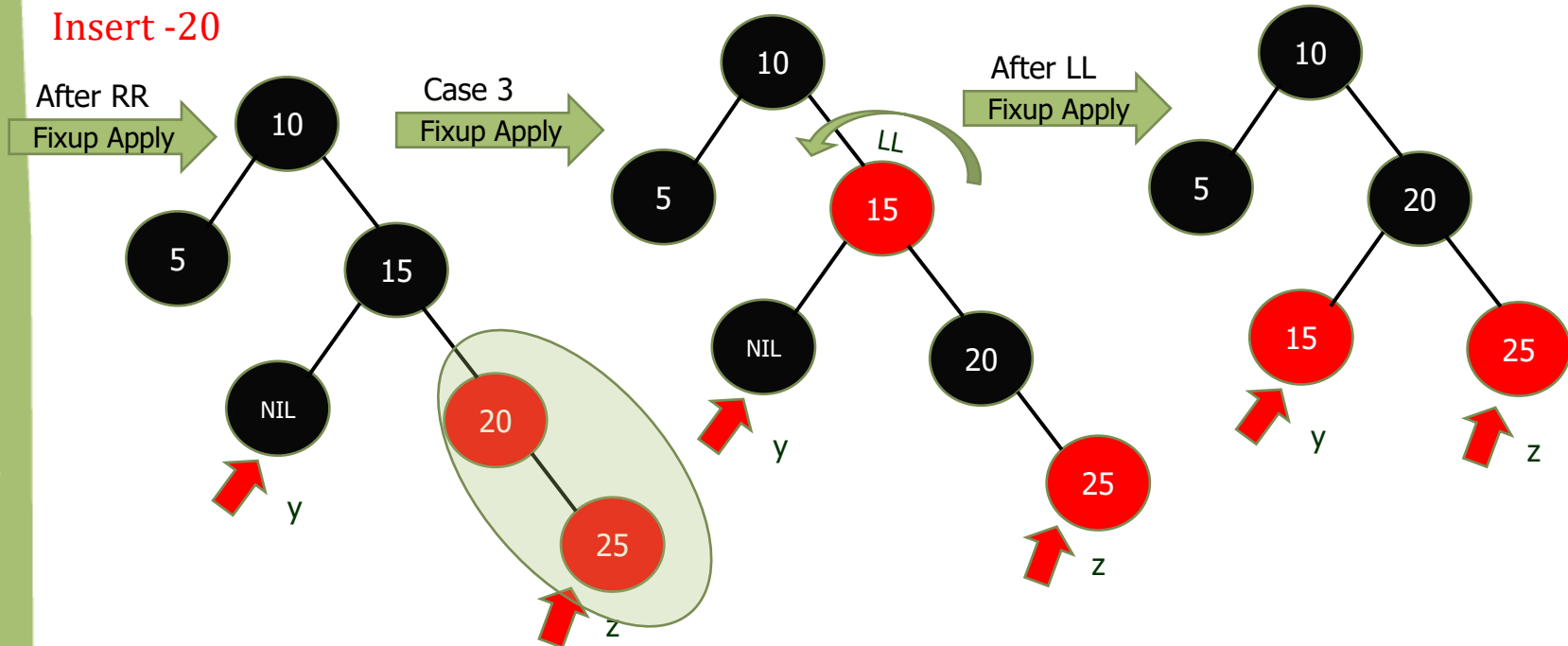
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -20



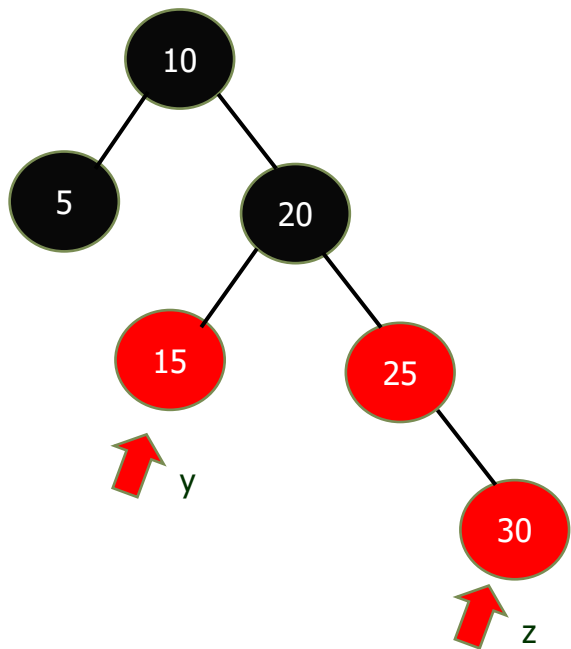
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -30



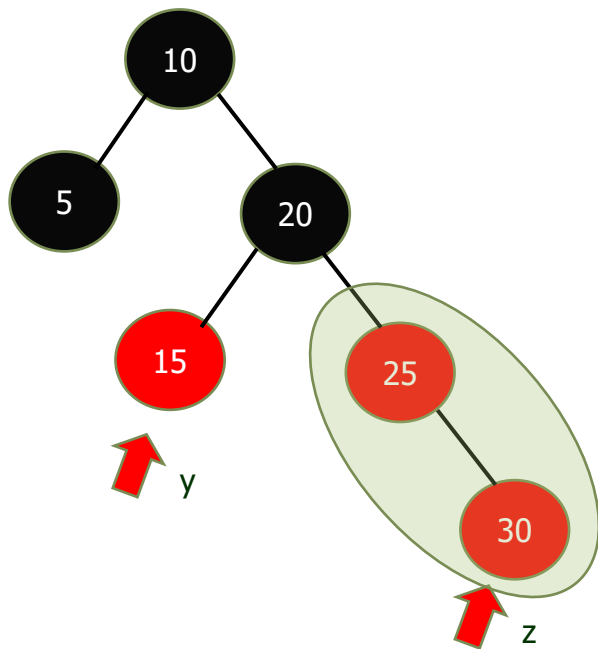
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -30



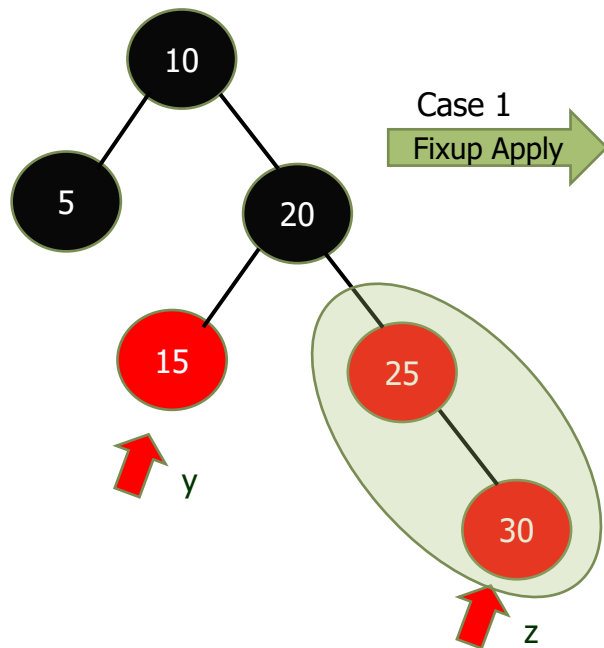
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -30



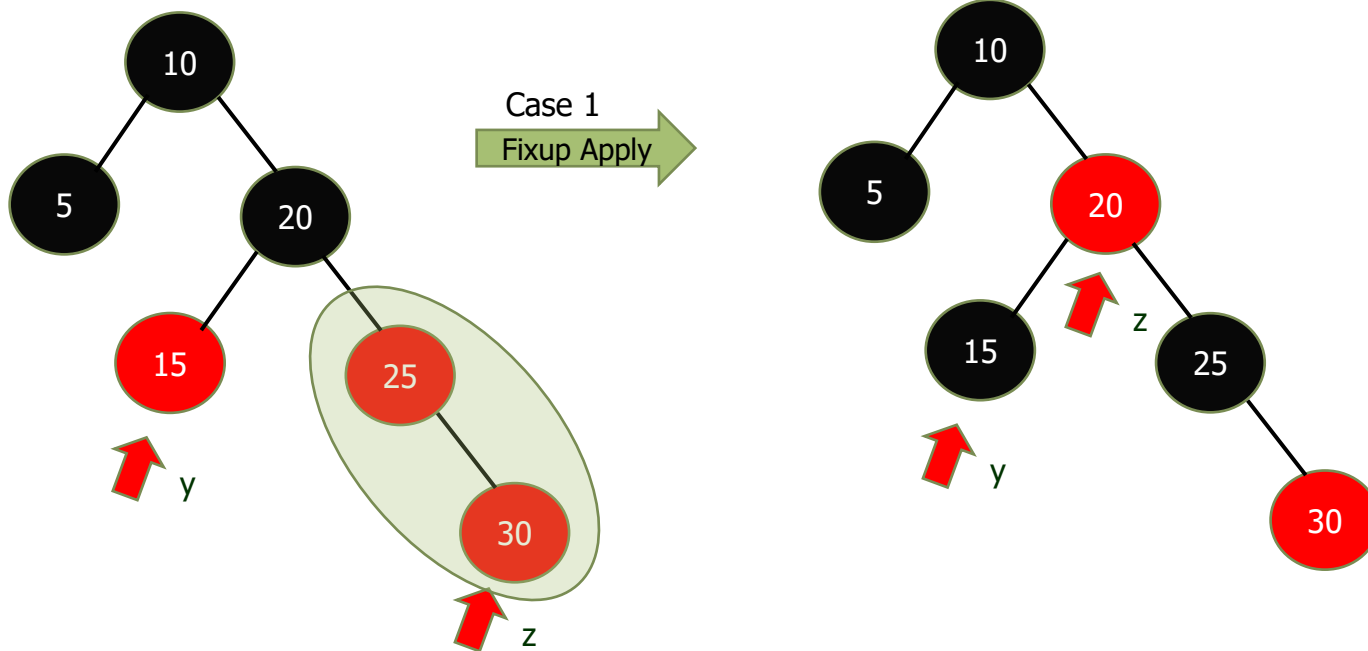
Red Black Tree

Insertion (Example 4):

Insert the following elements into an empty RB-Tree.

[5, 10, 15, 25, 20, 30]

Insert -30



Thank u